目录

板子整理（C++17） 3
0. 通用头 3
对拍 3
一、基础 4
1. 排序、离散化、二分 4
2. set / multiset / map 4
3. stack / queue / priority_queue / bitset 4
4.pd_ds 4
5. 异或与线性基 5
二、字符串 7
1. KMP 7
2. exKMP / Z 函数 7
3. Trie 树 8
4. AC 自动机 9
5. 字符串 Hash 10
6. Manacher 10
7. 最小表示法 11
8. 后缀数组 SA + LCP 11
9. 后缀自动机 SAM 12
10. 回文自动机 PAM 14
三、图论 14
1. 拓扑排序 14
2. Dijkstra 15
3. SPFA、判负环、差分约束 15
4. Tarjan 强连通分量 / 缩点 16
5. 双连通分量 17
6. 2-SAT 18
7. 最小生成树 Kruskal / Prim 19
8. 欧拉图 20
9. 二分图最大匹配：匈牙利 21
10. 二分图最大权完美匹配：KM 22
11. 最大流 Dinic 23
12. 最小费用最大流 24
13. 有上下界网络流 25
四、树与树上算法 26
1. LCA：倍增 26
2. 树上差分 26
3. 树链剖分 29
4. 树的直径、重心 30
5. 点分治 31
五、数据结构 32
1. 并查集 32
2. 树状数组 BIT 32
3. 线段树 (扫描线) 32
4. 动态开点线段树 33
5. 可持久化线段树（主席树） 34
6. FHQ Treap 34
7. 单调栈 / 单调队列 35
8. ST 表 36
9.莫队 36
六、数论与组合数学 36
1. gcd、扩展欧几里得、逆元 36
2. 矩阵快速幂 37
3. 欧拉筛 37

4. Miller-Rabin 素性检测（64 位整数） 37
5. 中国剩余定理 CRT 39
6. 组合数、Lucas 39
7. BSGS / exBSGS 40
8. 莫比乌斯函数与反演 40
9. 常用反演总览 40
10. FFT/NTT 44
11. FWT 45
七、动态规划 DP 46
1. 线性 DP / LIS 46
2. 背包 DP 46
3. 区间 DP 46
4. 树形 DP 47
5. 状压 DP 48
6. 数位 DP 48
7. DAG 上 DP 49
8. 单调队列优化 DP 49
9. 斜率优化 DP（CHT） 50
八、博弈论 50
1. Bash Game(巴什博弈) 50
2. Wythoff Game(威佐夫博弈) 50
3. Nim Game 51
4. SG 函数 51
九、计算几何 51
1. 向量 51
2.点和线 52
3. 多边形 53
补充： 55
1. Matrix-Tree 定理 (生成树的计数) 55
2. 高斯消元 56

板子整理（C++17）

0. 通用头

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
using pii = pair<int,int>;
using pll = pair<long long,long long>;
const int INF = 0x3f3f3f3f;
const long long LINF = (1LL<<62);
const int MOD = 1e9 + 7;
```

对拍

```
@echo off
echo Compiling...
g++ rnd.cpp -o rnd.exe -O2 -std=c++17
g++ bf.cpp -o bf.exe -O2 -std=c++17
g++ my.cpp -o my.exe -O2 -std=c++17
if errorlevel 1 (
    echo Compilation failed!
    pause
    exit
)
echo Compilation successful!
:: 对拍循环
set /a cnt=0
:loop
set /a cnt+=1
echo Testing case %cnt%...
:: 生成数据
rnd.exe > data.in
:: 运行两个程序
bf.exe < data.in > std.out
my.exe < data.in > my.out
:: 比较输出
fc std.out my.out > nul
if errorlevel 1 (
    echo Found different output at case %cnt%!
    pause
    exit
) else (
    echo Case %cnt% passed.
)
goto loop
```

一、基础

1. 排序、离散化、二分

```
vector<int> a = {5, 2, 2, 9};
sort(a.begin(), a.end());
a.erase(unique(a.begin(), a.end()), a.end());

int pos = lower_bound(a.begin(), a.end(), 7) - a.begin(); // 第一个 >= 7
int pos2 = upper_bound(a.begin(), a.end(), 7) - a.begin(); // 第一个 > 7
```

2. set / multiset / map

```
set<int> s;
s.insert(3); s.insert(1);
auto it = s.lower_bound(2); // >= 2
if(it != s.end()) cout << *it << '\n';

multiset<int> ms;
ms.insert(5); ms.insert(5);
ms.erase(ms.find(5)); // 只删一个 5
```

3. stack / queue / priority_queue / bitset

```
stack<int> st;
queue<int> q;
priority_queue<int> big; // 大根堆
priority_queue<int, vector<int>, greater<int>> small; // 小根堆
bitset<1005> b;
b.set(3); b.reset(3); b.flip(4);
int cnt = b.count();
```

4.pd_ds

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

gp_hash_table<int,int>mp;//和 map 作用相同但是非常快

__gnu_pbds::priority_queue<int,cmp>q,q1;//默认大根堆，配对堆,cmp可不写
__gnu_pbds::priority_queue<int>::point_iterator id=q.push(1);  // 点类型迭代器
// 在 modify 和 push 的时候都会返回一个 point_iterator
q.modify(id,2);//logn 修改 id 指向的点的值
q.erase(id);//logn 删除 id 指向的点，可以不是推顶
q.join(q1);//O1 把 q1 合并进 q，并把 q1 清空

tree<pii,null_type,less<pii>>tr;//默认红黑树，元素递增（less）
tr.insert({1,0});//插入，返回 pair<迭代器，bool是否插入成功>
tr.erase(x);//删除迭代器/元素x
tr.order_of_key({2,0});//返回小于 x 的元素个数，即从 0 开始的排名
tr.find_by_order(x);//返回排名为 x 的元素的迭代器
```

```
//tr.lower_bound和upper_bound 用法和set相同
tr.join(x);//把 x 树并入，并把 x 清空，不过需要保证两棵树值域无交集
tr.split(x,b);//小于等于 x 的元素属于当前树，其余的属于树 b
```

5. 异或与线性基

`1` 到 `n` 的异或和有 `4` 个一循环：

```
unsigned long long xor1ToN(unsigned long long n){
    switch(n&3){
        case 0:return n;
        case 1:return 1;
        case 2:return n+1;
        default:return 0;
    }
}
```

线性基

把每个整数看成一个二进制向量，整数异或就是向量加法。线性基是一组线性无关的向量，使原集合中所有数的异或组合都能由这组向量表示。

设线性基的秩为 `r`：

- 插入一个数后若秩增加，说明它不能由原线性基表示；
- 任意子集异或值都可以由线性基中的若干向量异或得到；
- 所有不同的子集异或值共有 `2^r` 个；
- 从最高位向最低位贪心，可以求出集合中任意子集异或的最大值；
- 对目标值从高位向低位消元，可以判断它是否能被表示。

```
struct LB {
    using ull = unsigned long long;
    static constexpr int LOG=64;
    ull b[LOG]{};
    int rank=0;
    bool insert(ull x){
        for(int i=LOG-1;i>=0;i--){
            if(((x>>i)&1)==0) continue;
            if(!b[i]){
                b[i]=x,rank++;
                return true;
            }
            x^=b[i];
        }
        return false;
    }
    bool canRepresent(ull x) const{
        for(int i=LOG-1;i>=0;i--){
            if(((x>>i)&1)==0) continue;
            if(!b[i]) return false;
            x^=b[i];
        }
        return true;
```

```
    }
    ull maxXor(ull x=0) const{
        for(int i=LOG-1;i>=0;i--)if((x^b[i])>x) x^=b[i];
        return x;
    }

    ull minXor(ull x) const{
        for(int i=LOG-1;i>=0;i--)if((x^b[i])<x) x^=b[i];
        return x;
    }
    void mergeFrom(const LinearBasis& other){
        for(ull x:other.b) if(x) insert(x);
    }
};
```

插入和查询的时间复杂度均为 `O(LOG)`，其中 `LOG=64`；合并两个线性基的复杂度为 `O(LOG^2)`，空间复杂度为 `O(LOG)`。

不同子集异或值的数量

若原数组中有 `r` 个线性无关的数，则不同子集异或值的数量为 `2^r`。当 `r<64` 时可直接写成 `1ULL<<r`；如果答案可能达到 `2^64`，应使用更大的整数类型保存。

```
u128 countDistinctSubsetXor(const vector<ull>& a){
    LinearBasis lb;
    for(auto x:a) lb.insert(x);
    return (u128)1<<lb.rank;
}
```

第 K 小子集异或值（包含空集异或值 `0`）

先把线性基规范化，使高位基向量消去低位基向量，再按主元从低位到高位排列。`k` 从 `0` 开始，返回所有不同子集异或值的第 `k` 小值。

```
pair<bool,ull> kthSubsetXor(const LB& basis,ull k){//bool存是否有第k个，ull存答案
    LB lb=basis;
    for(int i=0;i<LB::LOG;i++)
        if(lb.b[i])
            for(int j=i-1;j>=0;j--)
                if(lb.b[j] && ((lb.b[i]>>j)&1)) lb.b[i]^=lb.b[j];
    vector<ull> v;
    for(int i=0;i<LB::LOG;i++) if(lb.b[i]) v.push_back(lb.b[i]);
    u128 total=(u128)1<<v.size();
    if((u128)k>=total) return pair<bool,ull>{0,0};
    ull ans=0;
    for(int i=0;i<v.size();i++)
        if((k>>i)&1) ans^=v[i];
    return pair<bool,ull>{1,ans};
}
```

注意：线性基适合求“若干数任意选取后的异或”，而原数组中“任选两个数的最大异或”通常使用 `01-Trie`。

二、字符串

1. KMP

前缀函数与匹配

前缀函数 pi 定义：

- 如果子串 $s_{1\dots i}$ 有一对相等的真前缀与真后缀：$s_{1\dots k} = s_{i-k+1\dots i}$，那么 pi 就是这个相等的真前缀（或者真后缀）的**最长**长度，也就是 $pi = k$；

```
vector<int> kmp(const string& s){//注意字符串下标从0开始
    int n = s.size();
    vector<int> pi(n);
    for(int i=1;i<n;i++){
        int j = pi[i-1];
        while(j > 0 && s[i] != s[j]) j = pi[j-1];
        if(s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
for(int i=2,j=0;i<s.size();i++){//下标1开始
    while(j&&s[i]!=s[j+1])j=pi[j];
    pi[i]=j+=s[i]==s[j+1];
}
/*
两个串匹配可以把两个串拼起来跑kmp：
string s = pat + '#' + text;
从 pat.size()+1 位置找 pi[i]==pat.size()的位置
*/
```

求最短循环节

思路：若 `n % (n-pi[n-1]) == 0`，循环节长度为 `n-pi[n-1]`，否则整串是最短循环节。

```
int minPeriod(const string& s){
    auto pi = prefixFunction(s);
    int n = s.size(), p = n - pi[n-1];
    return (n % p == 0) ? p : n;
}
```

找前缀子串最长周期是求最短匹配，直接从 j=i 跳 j=pi[j]，跳到最小的 pi，最长周期是 i-j

2. exKMP / Z 函数

$z[i]$ 表示 s 和 $s_{i\dots n-1}$（即以 $s[i]$ 开头的后缀）的最长公共前缀（LCP）的长度

```cpp
vector<int> zF(const string& s){
    int n = s.size();
    vector<int> z(n);
    for(int i=1,l=0,r=0;i<n;i++){
        if(i <= r) z[i] = min(r-i+1, z[i-l]);
        while(i+z[i] < n && s[z[i]] == s[i+z[i]]) z[i]++;
        if(i+z[i]-1 > r) l = i, r = i+z[i]-1;
    }
    z[0] = n;
    return z;
}
```

经典应用：每个后缀与模式串的 LCP

思路：对 `pat + '#' + text` 求 Z，`z[m+1+i]` 就是 `text[i..]` 与 `pat` 的最长公共前缀。

```cpp
vector<int> lcpWithPattern(const string& text, const string& pat){
    string s = pat + '#' + text;
    auto z = zF(s);
    vector<int> ans(text.size());
    for(int i=0;i<(int)text.size();i++) ans[i] = min((int)pat.size(),
z[pat.size()+1+i]);
    return ans;
}
```

3. Trie 树

```cpp
struct Trie {
    struct Node { int nxt[26]; int cnt=0, end=0; Node(){ memset(nxt,0,sizeof
nxt); } };
    vector<Node> tr;
    Trie(){ tr.push_back(Node()); }
    void insert(const string& s){
        int u=0;
        for(char c:s){
            int x=c-'a';
            if(!tr[u].nxt[x]) tr[u].nxt[x]=tr.size(), tr.push_back(Node());
            u=tr[u].nxt[x]; tr[u].cnt++;
        }
        tr[u].end++;
    }
    bool search(const string& s){
        int u=0;
        for(char c:s){ int x=c-'a'; if(!tr[u].nxt[x]) return false;
u=tr[u].nxt[x]; }
        return tr[u].end > 0;
    }
};
```

经典应用：01 Trie 求最大异或

思路：从高位到低位贪心走相反位。

```
struct BinaryTrie {
    struct Node { int ch[2]{0,0}; };
    vector<Node> tr{Node()};
    void insert(int x){
        int u=0;
        for(int i=30;i>=0;i--){
            int b=(x>>i)&1;
            if(!tr[u].ch[b]) tr[u].ch[b]=tr.size(), tr.push_back(Node());
            u=tr[u].ch[b];
        }
    }
    int maxXor(int x){
        int u=0, ans=0;
        for(int i=30;i>=0;i--){
            int b=(x>>i)&1;
            if(tr[u].ch[b^1]) ans|=1<<i, u=tr[u].ch[b^1];
            else u=tr[u].ch[b];
        }
        return ans;
    }
};
```

4. AC 自动机

fail 指针：**节点 u** 的 fail 指针 指向另一个 节点 v，**v 所代表的前缀 是 u 所代表的前缀的最长后缀**

```
struct AhoCorasick {
    struct Node { int nxt[26]; int fail=0, out=0; Node(){ memset(nxt,0,sizeof
nxt); } };
    vector<Node> tr{Node()};
    void insert(const string& s){
        int u=0;
        for(char c:s){
            int x=c-'a';
            if(!tr[u].nxt[x]) tr[u].nxt[x]=tr.size(), tr.push_back(Node());
            u=tr[u].nxt[x];
        }
        tr[u].out++;
    }
    void build(){
        queue<int> q;
        for(int c=0;c<26;c++) if(tr[0].nxt[c]) q.push(tr[0].nxt[c]);
        while(!q.empty()){
            int u=q.front(); q.pop();
            int Fail=tr[u].fail;
            tr[u].out += tr[Fail].out;
            for(int c=0;c<26;c++){
                int v=tr[u].nxt[c];
                if(v) tr[v].fail = tr[Fail].nxt[c], q.push(v);
                else tr[u].nxt[c] = tr[Fail].nxt[c];
```

```
            }
        }
    }
    long long query(const string& s){
        int u=0; long long ans=0;
        for(char c:s){ u=tr[u].nxt[c-'a']; ans += tr[u].out; }
        return ans;
    }
};
```

多模式串出现次数

思路：把所有模式串插入 AC 自动机，文本每走一步，当前节点 fail 链上的终止标记都代表一次匹配。

```
long long countPatterns(vector<string> pats, string text){
    AhoCorasick ac;
    for(auto &p:pats) ac.insert(p);
    ac.build();
    return ac.query(text);
}
```

5. 字符串 Hash

```
ull h[N],pw[N];
inline ull get(int l,int r){return h[r]-h[l-1]*pw[r-l+1];}
inline ull get_s1_s2(int l, int r, int x, int y)//求子串[l,r]和[x,y]拼接的hash
{
    return get(l,r)*pw[y-x+1]+get(x,y);
}
void init(string s){    //自然溢出
    pw[0]=1;
    for(int i=1;i<int(s.size());i++)pw[i]=pw[i-1]*Mod;
    h[0]=1;
    for(int i=1;i<int(s.size());i++)
        h[i]=h[i-1]*Mod+s[i];
}
```

区间 [l,r] 是否有长度为 d 的循环节： 判断 $S[l+d \ldots r] == S[l \ldots r-d]$

6. Manacher

最长回文串长度

```
int len[N<<1];
inline int Manacher(const string &str)
{
    string s="~|";//边界
    for(char c:str)s+=c,s+='|';//保证串长为奇数
    int n=s.size()-1,res=0;
    for(int i=1,mid=1,r=1;i<=n;i++)
    {
        len[i]=min(len[mid-(i-mid)],r-i);
        while(len[i]<i&&i+len[i]<=n&&s[i-len[i]]==s[i+len[i]])++len[i];
```

```
        if(i+len[i]>r)r=i+len[mid=i];
        res=max(res,len[i]-1);
    }
    return res;
}
```

7. 最小表示法

与 S 循环同构的所有字符串中字典序最小的字符串

```
int minRotation(string s){
    int n=s.size(); s += s;
    int i=0,j=1,k=0;
    while(i<n && j<n && k<n){
        if(s[i+k] == s[j+k]) k++;
        else if(s[i+k] > s[j+k]) i = i+k+1, k=0, i += (i==j);
        else j = j+k+1, k=0, j += (i==j);
    }
    return min(i,j);
}
```

判断 a,b 循环同构

```
bool cyclicEqual(string a, string b){
    if(a.size()!=b.size()) return false;
    int pa=minRotation(a), pb=minRotation(b);
    rotate(a.begin(), a.begin()+pa, a.end());
    rotate(b.begin(), b.begin()+pb, b.end());
    return a==b;
}
```

8. 后缀数组 SA + LCP

$sa[i]$ 表示将所有后缀排序后第 i 小的后缀的编号。

$rk[i]$ 表示后缀 i $(s_{i...n})$ 的排名。

$height[i] = LCP(sa[i], sa[i-1])$，即第 i 名的后缀与它前一名的后缀的最长公共前缀。

```
int rk[N<<1],oldrk[N],sa[N<<1],id[N],cnt[N],height[N];
void SA(string s){//注意下标从 1 开始，s='~'+s
    int n=s.size()-1;
    int m=300,p=0;//m是不同字符个数
    for(int i=1;i<=m;i++)cnt[i]=0;//多测清空
    for(int i=1;i<=n;i++) cnt[rk[i]=s[i]]++;
    for(int i=1;i<=m;i++) cnt[i]+=cnt[i-1];
    for(int i=n;i>=1;i--) sa[cnt[rk[i]]--]=i;
    for(int w=1;;w<<=1,m=p) {  // m=p 即为值域优化
        int cur=0;
        for(int i=n-w+1;i<=n;i++) id[++cur]=i;
        for(int i=1;i<=n;i++)
        if(sa[i]>w) id[++cur]=sa[i]-w;
        memset(cnt,0,sizeof(cnt));
        for(int i=1;i<=n;i++) cnt[rk[i]]++;
```

```
        for(int i=1;i<=m;i++) cnt[i]+=cnt[i-1];
        for(int i=n;i>=1;i--) sa[cnt[rk[id[i]]]--]=id[i];
        p=0;
        memcpy(oldrk,rk,sizeof(oldrk));
        for(int i=1;i<=n;i++){
            if(oldrk[sa[i]]==oldrk[sa[i-1]]&&oldrk[sa[i]+w]==oldrk[sa[i-1]+w])
                rk[sa[i]]=p;
            else rk[sa[i]]=++p;
        }
        if(p==n) break; // p=n 时无需再排序
    }
    for(int i=1,k=0;i<=n;++i) {
        if(rk[i]==0) continue;
        if(k)--k;
        while(s[i+k]==s[sa[rk[i]-1]+k])++k;
        height[rk[i]]=k;
    }
}
```

两子串最长公共前缀

$LCP(sa[i],sa[j])=\min \text{height}[i+1..j] \iff LCP(i,j)=\min \text{height}[\text{rk[i]}+1..\text{rk[j]}]$

比较一个字符串的两个子串的大小关系

假设需要比较的是 $A=S[a..b]$ 和 $B=S[c..d]$ 的大小关系.

若 $lcp(sa[a],sa[c])\geq min(|A|,|B|)$，$A<B \iff |A|<|B|$

否则，$A<B \iff rk[a]<rk[c]$.

不同子串数量

思路：所有后缀贡献长度 `n-sa[i]` 的前缀，减去与上一个后缀的公共前缀 `height[i]`。

答案：$n(n+1)/2-\sum_{i=2}^{n} height[i]$

```
long long calc(const string& s){
    SuffixArray SA(s);
    long long ans=0; int n=s.size();
    for(int i=0;i<n;i++) ans += n - SA.sa[i] - SA.height[i];
    return ans;
}
```

9. 后缀自动机 SAM

```
const int MAXLEN = 1e6 + 5;       // 字符串最大长度
const int MAXNODE = MAXLEN * 2;  // SAM 节点数最多是字符串长度的 2 倍
struct SuffixAutomaton {
    int ch[MAXNODE][26]; // DAG 转移边（假设字符集为小写字母 a-z）
    int link[MAXNODE];   // 后缀链接 (Parent 树的父节点)
    int len[MAXNODE];    // 该状态包含的最长子串长度
    int cnt[MAXNODE];    // 该状态的 endpos 集合大小（即子串在原串的出现次数）
    int sz, last;
    void init() {// 初始化 SAM
```

```
        for (int i = 1; i <= sz; i++) {
            memset(ch[i], 0, sizeof(ch[i]));
            link[i] = len[i] = cnt[i] = 0;
        }
        sz = 1;  // 1 号节点是起始节点 (Root)
        last = 1;
    }
    // 在 SAM 尾部添加一个字符
    void extend(char cc) {
        int c=cc-'a', cur = ++sz;
        len[cur] = len[last] + 1;
        cnt[cur] = 1;    // 每次实际扩展出的新节点，代表一个原串前缀，初始计数为 1
        int p = last;
        while (p > 0 && !ch[p][c]) {// 沿后缀链接往回跳，添加缺失的转移
            ch[p][c] = cur;
            p = link[p];
        }
        if (p == 0) {// 情况 1：一直跳到了虚拟根节点 0，说明这个字符是第一次出现
            link[cur] = 1;
        } else {
            int q = ch[p][c];
            if (len[q] == len[p] + 1) {// 情况 2：转移非常平滑，直接链接
                link[cur] = q;
            } else {// 情况 3：需要拆分节点 (Clone)
                int clone = ++sz;
                len[clone] = len[p] + 1;
                link[clone] = link[q];
                memcpy(ch[clone], ch[q], sizeof(ch[q])); // 复制转移边
                while (p > 0 && ch[p][c] == q) {// 将原来指向 q 的转移改指向上面的克隆
节点
                    ch[p][c] = clone;
                    p = link[p];
                }
                link[q] = link[cur] = clone;// 更新源节点和新节点的后缀链接
            }
        }
        last = cur;
    }
    int c_bucket[MAXNODE], pos[MAXNODE];
    // 计算每个状态对应的子串在原串中出现的总次数
    // 利用 O(N) 的基数排序（桶排），按 len 从长到短计算 Parent 树
    void build_cnt() {
        for (int i = 1; i <= sz; i++) c_bucket[i] = 0;// 清空桶
        for (int i = 1; i <= sz; i++) c_bucket[len[i]]++;// 统计每个长度的节点数
        for (int i = 1; i <= sz; i++) c_bucket[i] += c_bucket[i - 1];// 做前缀和，
分配排名
        for (int i = 1; i <= sz; i++) pos[c_bucket[len[i]]--] = i;// 按照排名将节点
存入 pos 数组
        for (int i = sz; i >= 1; i--) {// 按照 len 从大到小（拓扑序），把 cnt 累加给
Parent
            int u = pos[i];
            if (link[u] > 0) cnt[link[u]] += cnt[u];
        }
    }
} sam;
```

10. 回文自动机 PAM

```
struct PAM {
    struct Node { int next[26]{}, fail=0, len=0, cnt=0; };
    vector<Node> tr; string s; int last;
    PAM(){ init(); }
    void init(){ tr.assign(2, Node()); tr[0].len=0; tr[0].fail=1; tr[1].len=-1;
tr[1].fail=1; last=0; s.clear(); }
    int getFail(int x){
        int n=s.size();
        while(n-2-tr[x].len < 0 || s[n-2-tr[x].len] != s.back()) x=tr[x].fail;
        return x;
    }
    void add(char cc){
        s.push_back(cc); int c=cc-'a';
        int cur=getFail(last);
        if(!tr[cur].next[c]){
            Node node; node.len=tr[cur].len+2;
            int now=tr.size(); tr.push_back(node);
            tr[now].fail = (node.len==1) ? 0 : tr[getFail(tr[cur].fail)].next[c];
            tr[cur].next[c]=now;
        }
        last=tr[cur].next[c]; tr[last].cnt++;
    }
};
```

不同回文子串数量

```
int countDifferentPalindromes(const string& s){
    PAM pam;
    for(char c:s) pam.add(c);
    return (int)pam.tr.size() - 2;
}
```

三、图论

1. 拓扑排序

基模板

```
vector<int> topoSort(int n, vector<vector<int>>& g){
    vector<int> indeg(n+1), topo;
    for(int u=1;u<=n;u++) for(int v:g[u]) indeg[v]++;
    queue<int> q;
    for(int i=1;i<=n;i++) if(!indeg[i]) q.push(i);
    while(!q.empty()){
        int u=q.front(); q.pop(); topo.push_back(u);
        for(int v:g[u]) if(--indeg[v]==0) q.push(v);
    }
    return topo; // size < n 表示有环
}
```

DAG 最长路

```
vector<long long> dagLongestPath(int n, vector<vector<pair<int,int>>>& g, int s){
    vector<vector<int>> plain(n+1);
    for(int u=1;u<=n;u++) for(auto [v,w]:g[u]) plain[u].push_back(v);
    auto topo = topoSort(n, plain);
    vector<long long> dp(n+1, -LINF); dp[s]=0;
    for(int u:topo) if(dp[u] > -LINF/2)
        for(auto [v,w]:g[u]) dp[v]=max(dp[v], dp[u]+w);
    return dp;
}
```

2. Dijkstra

最短路径计数

```
pair<vector<ll>, vector<int>> dijkstraCount(int n, vector<vector<Edge>>& g, int
s){
    vector<ll> d(n+1, LINF); vector<int> cnt(n+1,0);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>, greater<pair<ll,int>>> pq;
    d[s]=0; cnt[s]=1; pq.push({0,s});
    while(!pq.empty()){
        auto [du,u]=pq.top(); pq.pop();
        if(du!=d[u]) continue;
        for(auto e:g[u]){
            ll nd=du+e.w;
            if(nd<d[e.to]) d[e.to]=nd, cnt[e.to]=cnt[u], pq.push({nd,e.to});
            else if(nd==d[e.to]) cnt[e.to]=(cnt[e.to]+cnt[u])%MOD;
        }
    }
    return {d,cnt};
}
```

3. SPFA、判负环、差分约束

```
vector<long long> spfa(int n, vector<vector<Edge>>& g, int s){
    vector<ll> d(n+1, LINF); vector<int> inq(n+1);
    queue<int> q; d[s]=0; q.push(s); inq[s]=1;
    while(!q.empty()){
        int u=q.front(); q.pop(); inq[u]=0;
        for(auto e:g[u]) if(d[e.to] > d[u] + e.w){
            d[e.to] = d[u] + e.w;
            if(!inq[e.to]) inq[e.to]=1, q.push(e.to);
        }
    }
    return d;
}
```

判负环

```
bool hasNegativeCycle(int n, vector<vector<Edge>>& g){
    vector<ll> d(n+1,0); vector<int> cnt(n+1,0), inq(n+1,1);
    queue<int> q; for(int i=1;i<=n;i++) q.push(i);
    while(!q.empty()){
        int u=q.front(); q.pop(); inq[u]=0;
        for(auto e:g[u]) if(d[e.to] > d[u] + e.w){
            d[e.to] = d[u] + e.w;
            if(++cnt[e.to] >= n) return true;
            if(!inq[e.to]) inq[e.to]=1, q.push(e.to);
        }
    }
    return false;
}
```

差分约束

思路：约束 $x_v - x_u <= w$ 建边 `u -> v (w)`，求最短路；若有负环则无解。

```
bool differenceConstraints(int n, vector<tuple<int,int,int>> cons, vector<ll>& x)
{
    vector<vector<Edge>> g(n+1);
    for(auto [u,v,w]: cons) g[u].push_back({v,w});
    for(int i=0;i<n;i++) g[0].push_back({i+1,0}); // 超级源点
    if(hasNegativeCycle(n, g)) return false;
    x = spfa(n+1, g, 0);
    return true;
}
```

4. Tarjan 强连通分量 / 缩点

```
int n,m;
int tim=0,sccCnt=0;
vector<int>g[N];
int dfn[N],low[N],in[N],bel[N];
vector<int>st;
void tarjan(int u){
    dfn[u]=low[u]=++tim; st.push_back(u); in[u]=1;
    for(int v:g[u]){
        if(!dfn[v]) tarjan(v),low[u]=min(low[u],low[v]);
        else if(dfn[v]<dfn[u]) low[u]=min(low[u],dfn[v]);
    }
    if(dfn[u]==low[u]){//强连通分量缩点
        ++sccCnt;
        while(true){
            int x=st.back();st.pop_back();
            in[x]=0;bel[x]=sccCnt;
            if(x==u)break;
        }
    }
}
void solve(){
```

```
    for(int i=1;i<=n;i++) if(!dfn[i])tarjan(i);
}
```

5. 双连通分量

无向图 割点和点双

```
int n,m;
vector<Pii>g[N];
int dfn[N],low[N],iscut[N],tim,top;
vector<int>ans[N];//点双点集
stack<int>st;
void tarjan(int u,int in_edge){//放入边的编号，适用重边
    dfn[u]=low[u]=++tim; st.emplace(u); int son=0;
    for(auto[v,id]:g[u]){
        if(id==in_edge)continue;
        if(!dfn[v]){
            tarjan(v,id); son++;low[u]=min(low[u],low[v]);
            if(low[v]>=dfn[u]){//判断点双
                top++;
                while(true){
                    int p=st.top();st.pop(); ans[top].emplace_back(p);
                    if(p==v)break;
                }
                ans[top].emplace_back(u);//加入割点自己
                if(in_edge!=0)iscut[u]=1;
            }
        }
        else if(dfn[v]<dfn[u])low[u]=min(low[u],dfn[v]);
    }
    if(in_edge==0&&son>1)iscut[u]=1;
    if(in_edge==0&&son==0){
        ans[++top].emplace_back(u);//判断孤点
        st.pop();
    }
}
for(int i=1;i<=n;i++)if(!dfn[i])tarjan(i,0);
```

> **Note**
>
> - 一个构成点双的图，恰好可以划分出 $\lfloor deg_i/2 \rfloor$ 个经过 i 的边不相交的简单环。

边双连通分量

思路：先求割边，删除所有割边后求连通块，每个连通块就是一个边双。桥连接这些块形成一棵森林。

```
int n,m;
vector<Pii>g[N];
int dfn[N],low[N],tim,top;
bool isbridge[M];
vector<int>ans[N];//边双点集
void tarjan(int u,int inEdge){//用边的编号判返回边，防止该点和父亲有重边导致漏判割边
    dfn[u]=low[u]=++tim;
    for(auto[v,id]:g[u]){
```

```
        if(id==inEdge)continue;
        if(!dfn[v]){
            tarjan(v,id);
            low[u]=min(low[u],low[v]);
            if(low[v]>dfn[u])isbridge[id]=1;//割边
        }
        else low[u]=min(low[u],dfn[v]);
    }
}
bool vis[N];
void dfs(int u){
    vis[u]=1;ans[top].emplace_back(u);
    for(auto[v,id]:g[u]){
        if(vis[v]||isbridge[id])continue;
        dfs(v);
    }
}
for(int i=1;i<=n;i++) if(!dfn[i])tarjan(i,0);
for(int i=1;i<=n;i++) if(!vis[i])top++,dfs(i);
```

> **Note**
>
> - 边双连通具有传递性，即，若 x,y 边双连通， y,z 边双连通，则 x,z 边双连通。

6. 2-SAT

```
int n,m;
vector<int>g[N];//N要开两倍
void add(int u,int v){g[u].emplace_back(v);}
int tot;
stack<int>st;
int dfn[N],low[N],tim,bel[N];
bool vis[N];
void Tarjan(int u){
    dfn[u]=low[u]=++tim;
    st.push(u);vis[u]=1;
    for(int v:g[u]){
        if(!dfn[v]) Tarjan(v),low[u]=min(low[u],low[v]);
        else if(vis[v]) low[u]=min(low[u],dfn[v]);
    }
    if(dfn[u]==low[u]){
        int cur=0;++tot;
        do{
            cur=st.top();st.pop();
            vis[cur]=0;bel[cur]=tot;
        }while(cur!=u);
    }
}
int solve(){
    cin>>m>>n;
    for(int i=1;i<=m;i++){
        int a,b;cin>>a>>b;//±a 表示 a 选和不选
        int I=abs(a),J=abs(b),A=a>0?1:0,B=b>0?1:0;
        add(A?I+n:I,B?J:J+n);//i表示选，i+n表示不选
        add(B?J+n:J,A?I:I+n);
```

```
    }
    for(int i=1;i<=(n<<1);i++) if(!dfn[i])Tarjan(i);
    for(int i=1;i<=n;i++)
        if(bel[i]==bel[i+n])//无解
            return cout<<"No\n",0;
    vector<int>ans;
    cout<<"Yes\n";
    for(int i=1;i<=n;i++)
        if(bel[i]<bel[i+n])ans.emplace_back(i);//求得的 SCC 编号相当于反拓扑序，取小的
就是一组解
    cout<<ans.size()<<' ';
    for(int x:ans)cout<<x<<' ';
    cout<<'\n';
    return 0;
}
void clear(){//多测清空
    for(int i=1;i<=(n<<1);i++)g[i].clear();
    while(!st.empty())st.pop();
    tim=tot=0;
    memset(dfn+1,0,sizeof(int)*(2*n));
    memset(bel+1,0,sizeof(int)*(2*n));
    memset(vis+1,0,sizeof(bool)*(2*n));
}
```

2-SAT 建边需要把 形如二个变量至少满足一个的语句 变成两个 a **不成立**，则 b **成立** 语句的形式建边。

7. 最小生成树 Kruskal / Prim

Kruskal

```
struct MstDSU {
    vector<int> p, sz;
    MstDSU(int n=0){ init(n); }
    void init(int n){ p.resize(n+1); sz.assign(n+1,1); iota(p.begin(), p.end(),
0); }
    int find(int x){ return p[x]==x?x:p[x]=find(p[x]); }
    bool unite(int a,int b){
        a=find(a); b=find(b); if(a==b) return false;
        if(sz[a]<sz[b]) swap(a,b);
        p[b]=a; sz[a]+=sz[b]; return true;
    }
};

struct MstEdge { int u,v,w; };
long long kruskal(int n, vector<MstEdge> e){
    sort(e.begin(), e.end(), [](auto& a, auto& b){ return a.w < b.w; });
    MstDSU dsu(n); long long ans=0; int cnt=0;
    for(auto &x:e) if(dsu.unite(x.u,x.v)) ans += x.w, cnt++;
    return cnt==n-1 ? ans : -1;
}
```

Prim

```
int dis[N],vis[N];
int prim()//朴素，O(n^2)
{
    memset(dis,0x3f,sizeof dis);
    dis[1]=0;
    for(int j=1;j<=n;j++)
    {
        int min_len=2e9,k;
        for(int i=1;i<=n;i++)
            if(!vis[i]&&dis[i]<min_len)
                min_len=dis[i],k=i;
        vis[k]=1;
        for(int i=1;i<=n;i++)
            if(!vis[i]&&dis[i]>g[k][i])
                dis[i]=g[k][i];
    }
    int ans=0;
    for(int i=1;i<=n;i++)ans+=dis[i];
    return ans;
}
vector<pii>g[N];
long long prim(int n){//堆优化，O(mlogn)
    vector<int> vis(n+1); vector<ll> dis(n+1, LINF);
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    dis[1]=0; pq.push({0,1}); ll ans=0; int cnt=0;
    while(!pq.empty()){
        auto [d,u]=pq.top(); pq.pop();
        if(vis[u]) continue;
        vis[u]=1; ans+=d; cnt++;
        for(auto e:g[u]) if(!vis[e.to] && e.w<dis[e.to]) dis[e.to]=e.w,
pq.push({e.w,e.to});
    }
    return cnt==n ? ans : -1;
}
```

8. 欧拉图

欧拉路径是一条通过图中每条边**恰好一次**的路径

欧拉回路则是一条**起点和终点相同**的**欧拉路径**。

Hierholzer 求无向欧拉路径/回路

```
struct EulerUndirected {
    int n, m=0;
    vector<vector<pair<int,int>>> g;
    vector<int> used, path;
    EulerUndirected(int n): n(n), g(n+1) {}
    void addEdge(int u,int v){ g[u].push_back({v,m}); g[v].push_back({u,m});
used.push_back(0); m++; }
    void dfs(int u){
        while(!g[u].empty()){
```

```
            auto [v,id]=g[u].back(); g[u].pop_back();
            if(used[id]) continue;
            used[id]=1; dfs(v);
        }
        path.push_back(u);
    }
};
```

判断无向欧拉路径

思路：忽略孤立点后连通；奇度点数为 0 有欧拉回路，为 2 有欧拉路径。

```
bool hasEulerTrailUndirected(int n, vector<pair<int,int>> edges){
    vector<int> deg(n+1), p(n+1); iota(p.begin(), p.end(), 0);
    function<int(int)> find=[&](int x){ return p[x]==x?x:p[x]=find(p[x]); };
    auto unite=[&](int a,int b){ a=find(a); b=find(b); if(a!=b)p[a]=b; };
    for(auto [u,v]:edges) deg[u]++,deg[v]++,unite(u,v);
    int root=0, odd=0;
    for(int i=1;i<=n;i++) if(deg[i]){
        if(!root) root=find(i); else if(root!=find(i)) return false;
        odd += deg[i]&1;
    }
    return odd==0 || odd==2;
}
```

9. 二分图最大匹配：匈牙利

> 在二分图中找到一个边的集合，其中任意两条边都没有公共顶点，并且这个集合中的边数是最多的

```
struct Hungarian {
    int n, m;
    vector<vector<int>> g;
    vector<int> match, vis;
    Hungarian(int n,int m): n(n), m(m), g(n+1), match(m+1), vis(m+1) {}
    void addEdge(int u,int v){ g[u].push_back(v); }
    bool dfs(int u){
        for(int v:g[u]) if(!vis[v]){
            vis[v]=1;
            if(!match[v] || dfs(match[v])) return match[v]=u, true;
        }
        return false;
    }
    int solve(){//O(nm)
        int ans=0;
        for(int u=1;u<=n;u++){ fill(vis.begin(), vis.end(), 0); ans += dfs(u);
}//最大匹配边数
        return ans;
    }
};
```

最小点覆盖（Kőnig 定理）

> 用尽可能少的点去使所有的边都被选中

二分图最大匹配数 = 最小点覆盖数；

```
pair<vector<int>,vector<int>> minVertexCover(int n,int m,vector<vector<int>>&
g,vector<int>& matchR){
    vector<int> matchL(n+1), visL(n+1), visR(m+1);
    for(int r=1;r<=m;r++) if(matchR[r]) matchL[matchR[r]]=r;
    function<void(int)> dfs=[&](int u){
        visL[u]=1;
        for(int v:g[u]) if(matchL[u]!=v && !visR[v]){
            visR[v]=1;
            if(matchR[v] && !visL[matchR[v]]) dfs(matchR[v]);
        }
    };
    for(int u=1;u<=n;u++) if(!matchL[u]) dfs(u);
    vector<int> leftCover,rightCover;
    for(int u=1;u<=n;u++) if(!visL[u]) leftCover.push_back(u);
    for(int v=1;v<=m;v++) if(visR[v]) rightCover.push_back(v);
    return {leftCover,rightCover};//二分图左右部分的选点
}
```

10. 二分图最大权完美匹配：KM

> 在一个二分图中找到一组匹配，使得这组匹配中所有边的权值之和最大。

```
struct KM {
    int n;
    vector<vector<long long>> w;
    vector<long long> lx, ly;
    vector<int> matchY;
    KM(int n): n(n), w(n+1, vector<ll>(n+1)), lx(n+1), ly(n+1), matchY(n+1) {}
    long long solve(){
        for(int i=1;i<=n;i++) lx[i]=*max_element(w[i].begin()+1, w[i].end());
        for(int x=1;x<=n;x++){
            vector<ll> slack(n+1, LINF);
            vector<int> pre(n+1);
            vector<char> used(n+1);
            matchY[0]=x;// 0 号右点是本轮增广的虚点
            int y0=0;
            do{
                used[y0]=1;
                int x0=matchY[y0], y1=0;
                ll delta=LINF;
                for(int y=1;y<=n;y++) if(!used[y]){
                    ll d=lx[x0]+ly[y]-w[x0][y];
                    if(d<slack[y]) slack[y]=d,pre[y]=y0;
                    if(slack[y]<delta) delta=slack[y],y1=y;
                }
                for(int y=0;y<=n;y++){
                    if(used[y]) lx[matchY[y]]-=delta,ly[y]+=delta;
                    else slack[y]-=delta;
                }
```

```
                y0=y1;
            }while(matchY[y0]);
            do{
                int y1=pre[y0];
                matchY[y0]=matchY[y1];
                y0=y1;
            }while(y0);
        }
        ll ans=0; for(int y=1;y<=n;y++) ans += w[matchY[y]][y];
        return ans;
    }
};
```

时间复杂度 `O(n^3)`，空间复杂度 `O(n^2)`。

任务分配最大收益

思路：左部为人、右部为任务，`w[i][j]` 表示收益；若人数和任务数不同，补虚点权值 0。

```
long long assignmentMaxProfit(vector<vector<int>> profit){
    int n=profit.size(), m=profit[0].size(), N=max(n,m);
    KM km(N);
    for(int i=0;i<n;i++) for(int j=0;j<m;j++) km.w[i+1][j+1]=profit[i][j];
    return km.solve();
}
```

11. 最大流 Dinic

```
struct Dinic {
    struct E { int to, rev; long long cap; };
    int n; vector<vector<E>> g; vector<int> level, it;
    Dinic(int n): n(n), g(n+1), level(n+1), it(n+1) {}
    void addEdge(int fr,int to,ll cap){
        E a{to,(int)g[to].size(),cap}, b{fr,(int)g[fr].size(),0};
        g[fr].push_back(a); g[to].push_back(b);
    }
    bool bfs(int s,int t){
        fill(level.begin(), level.end(), -1); queue<int> q; level[s]=0;
q.push(s);
        while(!q.empty()){
            int u=q.front(); q.pop();
            for(auto &e:g[u]) if(e.cap>0 && level[e.to]<0)
level[e.to]=level[u]+1, q.push(e.to);
        }
        return level[t]>=0;
    }
    ll dfs(int u,int t,ll f){
        if(u==t) return f;
        for(int &i=it[u]; i<(int)g[u].size(); i++){
            E &e=g[u][i];
            if(e.cap>0 && level[e.to]==level[u]+1){
                ll ret=dfs(e.to,t,min(f,e.cap));
                if(ret){ e.cap-=ret; g[e.to][e.rev].cap+=ret; return ret; }
            }
```

```
        }
        return 0;
    }
    ll maxflow(int s,int t){
        ll flow=0, f;
        while(bfs(s,t)){
            fill(it.begin(), it.end(), 0);
            while((f=dfs(s,t,LINF))) flow+=f;
        }
        return flow;
    }
};
```

12. 最小费用最大流

Dijkstra + 势能版本（Primal-Dual）

```
struct MCMF {
    struct E { int to, rev; ll cap, cost; };
    int n; vector<vector<E>> g;
    MCMF(int n): n(n), g(n+1) {}
    void addEdge(int u,int v,ll cap,ll cost){
        E a{v,(int)g[v].size(),cap,cost}, b{u,(int)g[u].size(),0,-cost};
        g[u].push_back(a); g[v].push_back(b);
    }
    // 仅在初始残量网络存在负费用边时调用，求 Johnson 势能。
    // 前提：从 s 可达的残量网络中不存在负环。
    vector<ll> initialPotential(int s){
        vector<ll> h(n+1,LINF); h[s]=0;
        for(int it=1;it<n;it++){
            bool changed=false;
            for(int u=1;u<=n;u++) if(h[u]!=LINF){
                for(const E& e:g[u]) if(e.cap>0 && h[e.to]>h[u]+e.cost){
                    h[e.to]=h[u]+e.cost;
                    changed=true;
                }
            }
            if(!changed) break;
        }
        for(int i=1;i<=n;i++) if(h[i]==LINF) h[i]=0;
        return h;
    }
    pair<ll,ll> minCostMaxFlow(int s,int t,ll limit=LINF){//limit 限制最大流大小
        ll flow=0,cost=0;
        bool hasNegativeCost=false;
        for(int u=1;u<=n;u++) for(const E& e:g[u])
            if(e.cap>0 && e.cost<0) hasNegativeCost=true;
        vector<ll> h=hasNegativeCost?initialPotential(s):vector<ll>(n+1,0);
        vector<ll> d(n+1); vector<int> pv(n+1),pe(n+1);
        while(flow<limit){
            fill(d.begin(),d.end(),LINF);

 priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>> pq;
            d[s]=0; pq.push({0,s});
```

```
            while(!pq.empty()){
                auto [du,u]=pq.top(); pq.pop();
                if(du!=d[u]) continue;
                for(int i=0;i<(int)g[u].size();i++){
                    E &e=g[u][i];
                    ll nd=du+e.cost+h[u]-h[e.to];
                    if(e.cap>0 && d[e.to]>nd){
                        d[e.to]=nd; pv[e.to]=u; pe[e.to]=i;
                        pq.push({d[e.to],e.to});
                    }
                }
            }
            if(d[t]==LINF) break;
            for(int v=1;v<=n;v++) if(d[v]!=LINF) h[v]+=d[v];
            ll aug=limit-flow;
            for(int v=t;v!=s;v=pv[v]) aug=min(aug, g[pv[v]][pe[v]].cap);
            for(int v=t;v!=s;v=pv[v]){
                E &e=g[pv[v]][pe[v]]; e.cap-=aug; g[v][e.rev].cap+=aug;
            }
            flow+=aug;
            cost+=aug*(h[t]-h[s]);// 本次原图最短路费用
        }
        return {flow,cost};
    }
};
```

设 `V` 为点数、`E` 为残量网络边数、`A` 为增广次数，最大流 `F`，复杂度 `O(FE log V)`。若初始存在负费用边，额外进行一次 Bellman-Ford 初始化势能，复杂度为 `O(VE)`。空间复杂度为 `O(V+E)`。

最小费用匹配

思路：源点连左部容量 1，左部到右部容量 1 费用为代价，右部连汇点容量 1；跑最小费用最大流。

```
long long minCostBipartiteMatching(int n,int m,vector<tuple<int,int,int>> edges){
    int S=n+m+1,T=S+1; MCMF mf(T);
    for(int i=1;i<=n;i++) mf.addEdge(S,i,1,0);
    for(int j=1;j<=m;j++) mf.addEdge(n+j,T,1,0);
    for(auto [u,v,c]:edges) mf.addEdge(u,n+v,1,c);
    return mf.minCostMaxFlow(S,T).second;
}
```

13. 有上下界网络流

无源汇可行流

```
bool feasibleFlow(int n, vector<tuple<int,int,ll,ll>> edges){
    int S=n+1,T=n+2; Dinic dinic(T);
    vector<ll> demand(n+1);
    for(auto [u,v,low,cap]:edges){
        dinic.addEdge(u,v,cap-low);
        demand[u]-=low; demand[v]+=low;
    }
    ll need=0;
    for(int i=1;i<=n;i++){
```

```
        if(demand[i]>0) dinic.addEdge(S,i,demand[i]), need+=demand[i];
        else if(demand[i]<0) dinic.addEdge(i,T,-demand[i]);
    }
    return dinic.maxflow(S,T)==need;
}
```

经典应用：有源汇上下界最大流

思路：对原源汇加一条 `T -> S` 容量无限的边，转化为无源汇可行流；若可行，再在残量网络中从原源到原汇继续增广。

```
// 比赛实现建议让 Dinic::addEdge 返回边编号，方便查询原边流量。
// 建图转换：每条边 [low, cap] 变为 cap-low，同时维护 demand；额外加 t -> s 的 INF 边。
```

四、树与树上算法

1. LCA：倍增

```
struct LCA {
    int n, LOG, timer=0;
    vector<vector<int>> up, g;
    vector<int> dep, tin, tout;
    LCA(int n): n(n), LOG(32-__builtin_clz(n)), up(LOG, vector<int>(n+1)),
g(n+1), dep(n+1), tin(n+1), tout(n+1) {}
    void addEdge(int u,int v){ g[u].push_back(v); g[v].push_back(u); }
    void dfs(int u,int p){
        tin[u]=++timer; up[0][u]=p;
        for(int k=1;k<LOG;k++) up[k][u]=up[k-1][up[k-1][u]];
        for(int v:g[u]) if(v!=p) dep[v]=dep[u]+1, dfs(v,u);
        tout[u]=timer;
    }
    bool isAncestor(int a,int b){ return tin[a] <= tin[b] && tout[b] <= tout[a];
}
    int lca(int a,int b){
        if(isAncestor(a,b)) return a;
        if(isAncestor(b,a)) return b;
        for(int k=LOG-1;k>=0;k--) if(up[k][a] && !isAncestor(up[k][a],b)) a=up[k]
[a];
        return up[0][a];
    }
    int dist(int a,int b){ int c=lca(a,b); return dep[a]+dep[b]-2*dep[c]; }
};
```

2. 树上差分

点权差分：统计路径覆盖次数

思路：对于一条路径 `u -> v`，令 `w=LCA(u,v)`：

```
diff[u]++, diff[v]++;
diff[w]--;
diff[fa[w]]--;
```

最后从下往上累加 `diff`，得到每个点被多少条路径经过。

```
vector<int> pathCountOnTree(int n, vector<pair<int,int>> treeEdges,
vector<pair<int,int>> queries)
{
    LCA lca(n);
    for(auto [u,v]:treeEdges) lca.addEdge(u,v);
    lca.dfs(1,0);
    vector<int> diff(n+1), ans(n+1);
    for(auto [u,v]:queries)
    {
        int w=lca.lca(u,v);
        diff[u]++; diff[v]++; diff[w]--; diff[lca.up[0][w]]--;
    }
    function<void(int,int)> dfs=[&](int u,int p)
    {
        for(int v:lca.g[u]) if(v!=p) dfs(v,u),diff[u]+=diff[v];
        ans[u]=diff[u];
    };
    dfs(1,0);
    return ans;
}
```

点权差分：给路径整体加值

如果每条操作要求给路径 `u -> v` 上的所有点加 `x`，同样使用：

```
diff[u]+=x; diff[v]+=x;
diff[w]-=x; diff[fa[w]]-=x;
```

所有操作完成后进行一次 DFS 自底向上累加即可。

```
vector<long long> pathAddOnTree(int n, vector<pair<int,int>> treeEdges,
    vector<tuple<int,int,long long>> queries)
{
    LCA lca(n);
    for(auto [u,v]:treeEdges) lca.addEdge(u,v);
    lca.dfs(1,0);
    vector<ll> diff(n+1);
    for(auto [u,v,x]:queries)
    {
        int w=lca.lca(u,v);
        diff[u]+=x; diff[v]+=x;
        diff[w]-=x; diff[lca.up[0][w]]-=x;
    }
    function<void(int,int)> dfs=[&](int u,int p)
    {
        for(int v:lca.g[u]) if(v!=p) dfs(v,u),diff[u]+=diff[v];
    };
    dfs(1,0);
    return diff;
}
```

边权差分：统计每条边被路径覆盖次数

思路：把差分标记放在节点上，最后 DFS 累加。对于边 `(fa[u],u)`，它被覆盖的次数就是累加后的 `diff[u]`。

```
vector<int> edgeCountOnTree(int n, vector<pair<int,int>> treeEdges,
vector<pair<int,int>> queries)
{
    LCA lca(n);
    for(auto [u,v]:treeEdges) lca.addEdge(u,v);
    lca.dfs(1,0);
    vector<int> diff(n+1), ans(n+1);
    for(auto [u,v]:queries)
    {
        int w=lca.lca(u,v);
        diff[u]++; diff[v]++; diff[w]-=2;
    }
    function<void(int,int)> dfs=[&](int u,int p)
    {
        for(int v:lca.g[u]) if(v!=p)
        {
            dfs(v,u);
            diff[u]+=diff[v];
            ans[v]=diff[v];
        }
    };
    dfs(1,0);
    return ans;
}
```

树上差分公式总结

对于路径 `u -> v`，设 `w=LCA(u,v)`：

需求	差分操作
路径上的点	`d[u]++, d[v]++, d[w]--, d[fa[w]]--`
路径上的点加 `x`	`d[u]+=x, d[v]+=x, d[w]-=x, d[fa[w]]-=x`
路径上的边	`d[u]++, d[v]++, d[w]-=2`
路径上的边加 `x`	`d[u]+=x, d[v]+=x, d[w]-=2*x`

核心区别：

- **点差分**：`w` 和 `fa[w]` 都要处理。
- **边差分**：只需要 `w` 减 `2`。
- 最后都需要一次 **DFS 自底向上累加。**
- 树上差分适合「大量路径修改，最后统一统计」；如果要求修改后立即查询，通常使用 **树链剖分 + 线段树。**

---

3. 树链剖分

```
vector<int>e[N];
int fa[N],dep[N],hson[N],sz[N];
int dfn[N],top[N],val[N],cnt;
inline void dfs1(int u)
{
    dep[u]=dep[fa[u]]+1;
    hson[u]=-1,sz[u]=1;
    for(int v:e[u])
    {
        if(!dep[v])
        {
            fa[v]=u; dfs1(v); sz[u]+=sz[v];
            if(hson[u]==-1||sz[v]>sz[hson[u]])hson[u]=v;
        }
    }
}
inline void dfs2(int u,int Top)
{
    top[u]=Top; dfn[u]=++cnt; val[cnt]=u;
    if(hson[u]==-1)return;
    dfs2(hson[u],Top);
    for(int v:e[u])
        if(v!=fa[u]&&v!=hson[u])
            dfs2(v,v);
}
inline int LCA(int x,int y)
{
    int fx=top[x],fy=top[y];
    while(fx!=fy)
    {
        if(dep[fx]<=dep[fy]) y=fa[fy],fy=top[y];
        else x=fa[fx],fx=top[x];
    }
    return dep[x]<dep[y]?x:y;
}
```

树上区间问题

重链剖分把一条树上路径拆成 `O(log n)` 个 DFS 序连续区间，在同一条重链里的节点 x，y 的路径对应区间 $[dfn_x, dfn_y]$，点 x 的子树对应区间 $[dfn_x, dfn_x + size_x - 1]$，这可以用数据结构维护。

```
inline void Addxy(int x,int y,int z)//x,y 路径加 z
{
    int fx=top[x],fy=top[y];
    while(fx!=fy)
    {
        if(dep[fx]<dep[fy]) Modify(1,1,n,dfn[fy],dfn[y],z),y=fa[fy];
        else Modify(1,1,n,dfn[fx],dfn[x],z),x=fa[fx];
        fx=top[x];
        fy=top[y];
    }
    if(dfn[x]<dfn[y])
```

```
        Modify(1,1,n,dfn[x],dfn[y],z);
    else Modify(1,1,n,dfn[y],dfn[x],z);
}
```

4. 树的直径、重心

> **Note**
>
> - 若树上所有边边权均为正，则树的所有直径中点重合

```
int n,c,d[N];
vector<int>g[N];
void dfs(int u,int fa){//两次搜索求直径
    for(int v:g[u]){
        if(v==fa) continue;
        d[v]=d[u]+1;
        if(d[v]>d[c])c=v;
        dfs(v,u);
    }
}
void solve(){
    ...
    dfs(1,0);
    d[c]=0,dfs(c,0);//d[c]是直径长度
}
```

```
int n,d,dp[N];
void dfs(int u,int fa){//树形dp求直径
  for(int v:g[u]){
        if(v==fa)continue;
        dfs(v,u);
        d=max(d,dp[u]+dp[v]+1);
        dp[u]=max(dp[u],dp[v]+1);
    }
}//d是直径
```

求树重心

等价定义：

- 在树中删去结点 v 后，得到的图中每个连通分量的大小均不超过原树结点数的一半.
- 树中所有结点到某个结点的距离和中，到结点 v 的距离和最小.

> **Note**
>
> 1. 树的重心 如果 不唯一，则恰有两个．这两个重心相邻．而且，删去它们的连边后，树将变为两个大小相同的连通分量．
> 2. 在一棵树上添加或删除一个叶子，那么它的重心最多只移动一条边的距离．
> 3. 把两棵树通过一条边相连得到一棵新的树，那么新树的重心在连接原来两棵树的重心的路径上．
> 4. 一棵有根树的重心一定在根结点所在的重链上．一棵树的重心一定是该树根结点重子结点对应子树的重心的祖先．

```
int n,siz[N],weight[N];  // 所有子树「大小」的最大值
vector<int>centroids;  // 记录树的重心编号
vector<int>g[N];
void dfs(int u,int fa){
    siz[u]=1,weight[u]=0;
    for(int v:g[u])
        if(v!=fa){
            dfs(v,u);
            siz[u]+=siz[v];
            weight[u]=max(weight[u],siz[v]);
        }
    weight[u]=max(weight[u],n-siz[u]);
    if(weight[u]<=n/2)centroids.push_back(u);
}
```

5. 点分治

统计树上距离不超过 K 的点对

```
struct CentroidPairs {
    int n,K,ans=0;
    vector<vector<pair<int,int>>> g;
    vector<int> sz, dead;
    CentroidPairs(int n,int K): n(n),K(K),g(n+1),sz(n+1),dead(n+1) {}
    void addEdge(int u,int v,int w){ g[u].push_back({v,w});
g[v].push_back({u,w}); }
    void getSize(int u,int p){
        sz[u]=1;
        for(auto [v,w]:g[u]) if(v!=p&&!dead[v]) getSize(v,u),sz[u]+=sz[v];
    }
    int getCentroid(int u,int p,int tot){
        for(auto [v,w]:g[u]) if(v!=p&&!dead[v]&&sz[v]*2>tot) return
getCentroid(v,u,tot);
        return u;
    }
    void collect(int u,int p,int d,vector<int>& ds){
        if(d>K) return; ds.push_back(d);
        for(auto [v,w]:g[u]) if(v!=p&&!dead[v]) collect(v,u,d+w,ds);
    }
    int countPairs(vector<int>& a){
        sort(a.begin(),a.end()); int l=0,r=(int)a.size()-1,c=0;
        while(l<r){ if(a[l]+a[r]<=K) c+=r-l,l++; else r--; }
        return c;
    }
    void solve(int entry){
        getSize(entry,0); int c=getCentroid(entry,0,sz[entry]); dead[c]=1;
        vector<int> all{0};
        for(auto [v,w]:g[c]) if(!dead[v]){
            vector<int> ds; collect(v,c,w,ds);
            ans-=countPairs(ds); all.insert(all.end(),ds.begin(),ds.end());
        }
        ans+=countPairs(all);
        for(auto [v,w]:g[c]) if(!dead[v]) solve(v);
    }
```

```
};
int countPairsLeqK(int n,int K,vector<tuple<int,int,int>> edges){
    CentroidPairs cd(n,K);
    for(auto [u,v,w]:edges) cd.addEdge(u,v,w);
    cd.solve(1); return cd.ans;
}
```

五、数据结构

1. 并查集

带权并查集

```
struct WeightedDSU {
    vector<int> p, sz; vector<ll> dis;
    WeightedDSU(int n):p(n+1),sz(n+1,1),dis(n+1){iota(p.begin(),p.end(),0);}
    int find(int x){
        if(p[x]==x) return x;
        int f=p[x]; p[x]=find(p[x]); dis[x]+=dis[f]; return p[x];
    }
    bool unite(int a,int b,ll w){ // dis[b] - dis[a] = w
        int ra=find(a),rb=find(b); ll da=dis[a],db=dis[b];
        if(ra==rb) return db-da==w;
        if(sz[ra]<sz[rb]) swap(a,b),swap(ra,rb),swap(da,db),w=-w;
        p[rb]=ra; dis[rb]=da+w-db; sz[ra]+=sz[rb]; return true;
    }
};
```

2. 树状数组 BIT

> 单点加、前缀和

```
struct Fenwick {
    int n; vector<ll> t;
    Fenwick(int n=0):n(n),t(n+1){}
    void add(int x,ll v){for(;x<=n;x+=x&-x)t[x]+=v;}
    ll sum(int x){ll r=0;for(;x;x-=x&-x)r+=t[x];return r;}
    ll rangeSum(int l,int r){return sum(r)-sum(l-1);}
};
```

3. 线段树 (扫描线)

矩形面积并

思路：矩形拆成左右事件；离散化 y；线段树维护当前覆盖的 y 总长度，面积增量为 `当前覆盖长度 * 相邻 x 差`。

```
struct SweepSeg {//从左往右扫，线段树存 y 坐标线段
    vector<int> ys,cnt,len;
    SweepSeg(vector<int> y):ys(move(y)){
        int z=4*ys.size()+5;cnt.assign(z,0);len.assign(z,0);
```

```
    }
    void up(int p,int l,int r){
        if(cnt[p])len[p]=ys[r+1]-ys[l];
        else if(l==r)len[p]=0;
        else len[p]=len[p*2]+len[p*2+1];
    }
    void add(int p,int l,int r,int ql,int qr,int v){
        if(ql<=l&&r<=qr){cnt[p]+=v;up(p,l,r);return;}
        int m=(l+r)/2;
        if(ql<=m)add(p*2,l,m,ql,qr,v);
        if(qr>m)add(p*2+1,m+1,r,ql,qr,v);
        up(p,l,r);
    }
};
struct Seg{//矩阵拆成两个线段
    int x,y1,y2,t;//t 表示入边还是出边
};
long long calc(vector<array<int,4>> rec){//传入矩阵端点坐标
    vector<Seg> ev; vector<int> ys;
    for(auto [x1,y1,x2,y2]:rec){
        ev.push_back({x1,y1,y2,1});ev.push_back({x2,y1,y2,-1});
        ys.push_back(y1);ys.push_back(y2);
    }
    sort(ys.begin(),ys.end());ys.erase(unique(ys.begin(),ys.end()),ys.end());//离散化 y
    sort(ev.begin(),ev.end(),[](auto&a,auto&b){
        if(a.x==b.x)return a.t>b.t;
        return a.x<b.x;
    });//按 x 排序，相同的按入边先处理
    SweepSeg st(ys); long long ans=0;
    for(int i=0;i<(int)ev.size();i++){
        if(i)ans+=1LL*(ev[i].x-ev[i-1].x)*st.len[1];
        int l=lower_bound(ys.begin(),ys.end(),ev[i].y1)-ys.begin();
        int r=lower_bound(ys.begin(),ys.end(),ev[i].y2)-ys.begin()-1;//[l,r]在线段树里是[l,r-1],要-1
        if(l<=r)st.add(1,0,(int)ys.size()-2,l,r,ev[i].t);
    }
    return ans;
}
```

4. 动态开点线段树

```
struct DynamicSeg {
    struct Node{int l=0,r=0;ll sum=0,lazy=0;};
    vector<Node> tr{{}};
    int newNode(){tr.push_back(Node());return tr.size()-1;}
    void apply(int p,ll l,ll r,ll v){tr[p].sum+=v*(r-l+1);tr[p].lazy+=v;}
    void push(int p,ll l,ll r){
        if(!tr[p].l)tr[p].l=newNode(); if(!tr[p].r)tr[p].r=newNode();
        if(tr[p].lazy){ll m=(l+r)/2;apply(tr[p].l,l,m,tr[p].lazy);apply(tr[p].r,m+1,r,tr[p].lazy);tr[p].lazy=0;}
    }
    void add(int p,ll l,ll r,ll ql,ll qr,ll v){
        if(ql<=l&&r<=qr)return apply(p,l,r,v);
```

```
        push(p,l,r);ll m=(l+r)/2;
        if(ql<=m)add(tr[p].l,l,m,ql,qr,v);
        if(qr>m)add(tr[p].r,m+1,r,ql,qr,v);
        tr[p].sum=tr[tr[p].l].sum+tr[tr[p].r].sum;
    }
};
```

5. 可持久化线段树（主席树）

静态区间第 k 小

```
struct PersistentSeg {
    struct Node{int l=0,r=0,sum=0;};
    vector<Node> tr{{}}; vector<int> root,xs;
    int clone(int p){tr.push_back(tr[p]);return tr.size()-1;}
    int update(int p,int l,int r,int pos){
        int q=clone(p);tr[q].sum++;
        if(l!=r){int m=(l+r)/2;if(pos<=m)tr[q].l=update(tr[p].l,l,m,pos);else
tr[q].r=update(tr[p].r,m+1,r,pos);}
        return q;
    }
    int kth(int u,int v,int l,int r,int k){
        if(l==r)return l;
        int c=tr[tr[v].l].sum-tr[tr[u].l].sum,m=(l+r)/2;
        if(k<=c)return kth(tr[u].l,tr[v].l,l,m,k);
        return kth(tr[u].r,tr[v].r,m+1,r,k-c);
    }
    void build(vector<int> a){

 xs=a;sort(xs.begin(),xs.end());xs.erase(unique(xs.begin(),xs.end()),xs.end());ro
ot.assign(a.size()+1,0);
        for(int i=0;i<(int)a.size();i++){int
p=lower_bound(xs.begin(),xs.end(),a[i])-
xs.begin()+1;root[i+1]=update(root[i],1,xs.size(),p);}
    }
    int query(int l,int r,int k){return xs[kth(root[l-
1],root[r],1,xs.size(),k)-1];}
};
// PersistentSeg pst; pst.build(a);
// cout << pst.query(l,r,k) << '\n';
```

6. FHQ Treap

有序集合

```
struct FHQ {
    struct Node{int l=0,r=0,key,pri,sz=1;};
    vector<Node> tr{{}};mt19937
rng{chrono::steady_clock::now().time_since_epoch().count()};
    int sz(int p){return p?tr[p].sz:0;}
    void pull(int p){if(p)tr[p].sz=sz(tr[p].l)+sz(tr[p].r)+1;}
    int newNode(int key){tr.push_back({0,0,key,(int)rng(),1});return
tr.size()-1;}
```

```
    void split(int p,int key,int& a,int& b){
        if(!p){a=b=0;return;}
        if(tr[p].key<=key)a=p,split(tr[p].r,key,tr[p].r,b),pull(p);
        else b=p,split(tr[p].l,key,a,tr[p].l),pull(p);
    }
    int merge(int a,int b){
        if(!a||!b)return a|b;
        if(tr[a].pri<tr[b].pri){tr[a].r=merge(tr[a].r,b);pull(a);return a;}
        tr[b].l=merge(a,tr[b].l);pull(b);return b;
    }
    void insert(int& root,int x){
        int a,b;split(root,x,a,b);root=merge(merge(a,newNode(x)),b);
    }
    void erase(int& root,int x){
        int a,b,c;split(root,x,a,c);split(a,x-1,a,b);
        b=merge(tr[b].l,tr[b].r);root=merge(merge(a,b),c);
    }
};
```

动态排名

```
int rankOf(FHQ& t,int& root,int x){
    int a,b;t.split(root,x-1,a,b);int ans=t.sz(a)+1;root=t.merge(a,b);return ans;
}
```

7. 单调栈 / 单调队列

下一个更小元素

```
vector<int> nextLess(vector<int> a){
    int n=a.size();vector<int> ans(n,n),st;
    for(int i=0;i<n;i++){
        while(!st.empty()&&a[st.back()]>a[i])ans[st.back()]=i,st.pop_back();
        st.push_back(i);
    }
    return ans;
}
```

滑动窗口最大值

```
vector<int> slidingMax(vector<int> a,int k){
    deque<int> q;vector<int> ans;
    for(int i=0;i<(int)a.size();i++){
        while(!q.empty()&&q.front()<=i-k)q.pop_front();
        while(!q.empty()&&a[q.back()]<=a[i])q.pop_back();
        q.push_back(i);if(i>=k-1)ans.push_back(a[q.front()]);
    }
    return ans;
}
```

8. ST 表

可以维护区间 min/max/gcd

```
int lg[N];
vector<vector<int>>st;
void build(vector<int>a){
    int n=a.size();
    for(int i=2;i<=n;i++)lg[i]=lg[i/2]+1;
    st.assign(lg[n]+1,vector<int>(n));st[0]=a;
    for(int k=1;k<(int)st.size();k++)
        for(int i=0;i+(1<<k)<=n;i++)
            st[k][i]=min(st[k-1][i],st[k-1][i+(1<<(k-1))]);
}
int query(int l,int r){int k=lg[r-l+1];return min(st[k][l],st[k][r-(1<<k)+1]);}
```

9.莫队

```
struct Query{int l,r,id;};
vector<long long> mo(vector<int> a,vector<Query> qs){
    int n=a.size(),B=max(1,(int)sqrt(n));
    sort(qs.begin(),qs.end(),[&](Query x,Query y){
        int bx=x.l/B,by=y.l/B;if(bx!=by)return bx<by;
        return bx&1?x.r>y.r:x.r<y.r;
    });
    vector<ll> ans(qs.size());int L=0,R=-1;ll cur=0;
    auto add=[&](int pos){cur+=a[pos];};
    auto del=[&](int pos){cur-=a[pos];};
    for(auto q:qs){
        while(L>q.l)add(--L);while(R<q.r)add(++R);
        while(L<q.l)del(L++);while(R>q.r)del(R--);
        ans[q.id]=cur;
    }
    return ans;
}
```

六、数论与组合数学

1. gcd、扩展欧几里得、逆元

```
ll gcdll(ll a,ll b){return b?gcdll(b,a%b):abs(a);}
ll exgcd(ll a,ll b,ll& x,ll& y){
    if(!b){x=1;y=0;return a;}
    ll g=exgcd(b,a%b,y,x);y-=a/b*x;return g;
}
ll inv(ll a,ll mod){
    ll x,y,g=exgcd(a,mod,x,y);if(g!=1)return -1;
    x%=mod;if(x<0)x+=mod;return x;
}
```

解线性同余方程 `a x ≡ b (mod m)`

```
pair<ll,ll> solveCongruence(ll a,ll b,ll m){
    ll g=gcdll(a,m);if(b%g)return {-1,-1};
    a/=g;b/=g;m/=g;
    ll x=inv((a%m+m)%m,m);x=(__int128)x*(b%m)%m;
    return pair<ll,ll>{x,m}; // x + k*m
}
```

2. 矩阵快速幂

```
template<int N>
struct Matrix{
    ll a[N][N]{};
    Matrix operator*(const Matrix& o)const{
        Matrix r;
        for(int i=0;i<N;i++)for(int k=0;k<N;k++)if(a[i][k])for(int j=0;j<N;j++)
            r.a[i][j]=(r.a[i][j]+a[i][k]*o.a[k][j])%MOD;
        return r;
    }
};Matrix<n>;
```

3. 欧拉筛

```
vector<int>primes;
int lp[N],phi[N],mu[N];
void pri(int n){
    phi[1]=1,mu[1]=1;
    for(int i=2;i<=n;i++){
        if(!lp[i]){
            lp[i]=i,primes.push_back(i);
            phi[i]=i-1,mu[i]=-1;
        }
        for(int p:primes){
            if(p>lp[i]||1LL*i*p>n)break;
            lp[i*p]=p;
            if(i%p==0){phi[i*p]=phi[i]*p,mu[i*p]=0;break;}
            phi[i*p]=phi[i]*(p-1),mu[i*p]=-mu[i];
        }
    }
}
```

4. Miller-Rabin 素性检测（64 位整数）

```
using u64=uint64_t;using u128=__uint128_t;
ll modMul(ll a,ll b,ll mod){return (u128)a*b%mod;}
ll modPow(ll a,ll e,ll mod){
    ll r=1;for(;e;e>>=1,a=modMul(a,a,mod))if(e&1)r=modMul(r,a,mod);return r;
}

bool isPrime(ll n){
    if(n<2)return false;
```

```
    for(ll p:{2,3,5,7,11,13,17,19,23,29,31,37}){if(n%p==0)return n==p;}
    ll d=n-1,s=0;while(!(d&1))d>>=1,s++;
    for(ll a:{2LL,325LL,9375LL,28178LL,450775LL,9780504LL,1795265022LL}){
        if(a%n==0)continue;ll x=modPow(a%n,d,n);if(x==1||x==n-1)continue;
        bool ok=false;for(int r=1;r<s;r++){x=modMul(x,x,n);if(x==n-1)
{ok=true;break;}}
        if(!ok)return false;
    }
    return true;
}
```

Pollard-Rho 整数分解（64 位整数）

```
ll pollardRho(ll n){
    if(n%2==0) return 2;
    if(n%3==0) return 3;
    static mt19937_64 rng(time(0));
    while(true){
        ll c=uniform_int_distribution<ll>(1,n-1)(rng);
        ll x=uniform_int_distribution<ll>(0,n-1)(rng), y=x, d=1;
        auto f=[&](ll v){
            return (ll)(((u128)modMul(v,v,n)+c)%n);
        };
        while(d==1){
            x=f(x);
            y=f(f(y));
            d=gcdll(x>y?x-y:y-x,n);
        }
        if(d!=n) return d;
    }
}
void factorRec(ll n,vector<ll>& fac){
    if(n==1) return;
    if(isPrime(n)){
        fac.push_back(n);
        return;
    }
    ll d=pollardRho(n);
    factorRec(d,fac);
    factorRec(n/d,fac);
}
vector<ll> factorize64(ll n){//调用这个函数得到 n 的所有质因子 fac
    vector<ll> fac;
    factorRec(n,fac);
    sort(fac.begin(),fac.end());
    return fac;
}
```

该模板依赖 Miller-Rabin 中的 `u128 / modMul / isPrime`，以及前面的 `gcdll`。

5. 中国剩余定理 CRT

模数不互质 CRT

```
ll safeMod(ll x,ll m){x%=m;if(x<0)x+=m;return x;}
optional<pair<ll,ll>> exCRT(vector<ll> a, vector<ll> m){
    ll r=a[0], mod=m[0];
    for(int i=1;i<(int)a.size();i++){
        ll x,y,g=exgcd(mod,m[i],x,y);
        ll c=a[i]-r;
        if(c%g)return nullopt;
        ll t=mulMod(safeMod(c/g,m[i]/g), safeMod(x,m[i]/g), m[i]/g);
        r += mod*t;
        mod = mod/g*m[i];
        r = safeMod(r,mod);
    }
    return pair<ll,ll>{r,mod};
}
```

合并多个同余条件

```
// x ≡ a[i] (mod m[i])，若有解返回最小非负解 r 与 lcm 模数 mod。
```

6. 组合数、Lucas

预处理阶乘求组合数

```
vector<ll> fac, ifac;
void init(int n,int mod){
    fac.assign(n+1,1);ifac.assign(n+1,1);
    for(int i=1;i<=n;i++)fac[i]=fac[i-1]*i%mod;
    ifac[n]=qpow(fac[n],mod-2,mod);
    for(int i=n;i;i--)ifac[i-1]=ifac[i]*i%mod;
}
ll C(int n,int k,int mod){
    if(k<0||k>n)return 0;
    return fac[n]*ifac[k]%mod*ifac[n-k]%mod;
}
```

Lucas 定理（模数为质数）

```
ll lucas(ll n,ll k,int p){
    if(k<0||k>n)return 0;
    if(k==0)return 1;
    return lucas(n/p,k/p,p)*C(n%p,k%p,p)%p;
}
```

exLucas 思路：当模数不是质数，将模数质因数分解成 `p^q`，分别求 `C(n,m) mod p^q`，再用 CRT 合并。实现较长，关键是阶乘去掉所有 `p` 因子后递归计算。

7. BSGS / exBSGS

BSGS 求 `a^x ≡ b (mod p)`

```
ll BSGS(ll a,ll b,ll p){
    a%=p;b%=p;if(b==1%p)return 0;
    int m=ceil(sqrt((double)p));
    unordered_map<ll,int> table;
    ll e=1;
    for(int j=0;j<m;j++){if(!table.count(e))table[e]=j;e=e*a%p;}
    ll factor=qpow(a,p-1-m,p), cur=b;
    for(int i=0;i<=m;i++){
        if(table.count(cur))return 1LL*i*m+table[cur];
        cur=cur*factor%p;
    }
    return -1;
}
```

离散对数

思路：把指数拆成 `x = i*m + j`，预处理小步 `a^j`，大步用 `a^{-m}` 倒推。`gcd(a,p)!=1` 时用 exBSGS 先不断约掉公共因子。

```
// exBSGS: 若 gcd(a,p)=g，需 b%g==0；令 p/=g,b/=g，并累计系数，直到 a 与 p 互质后进入 BSGS。
```

8. 莫比乌斯函数与反演

求 `1<=i<=n,1<=j<=m` 且 `gcd(i,j)=1` 的对数

```
long long countCoprimePairs(int n,int m){
    int N=min(n,m);linearSieve(N);
    vector<long long> pre(N+1);
    for(int i=1;i<=N;i++)pre[i]=pre[i-1]+mu[i];
    long long ans=0;
    for(int l=1,r;l<=N;l=r+1){
        r=min(n/(n/l),m/(m/l));
        ans+=(pre[r]-pre[l-1])*(n/l)*(m/l);
    }
    return ans;
}
```

9. 常用反演总览

9.1 狄利克雷卷积

对算术函数 `f,g`，定义狄利克雷卷积：

$$(f*g)(n)=\sum_{d|n} f(d)g(n/d)$$

常用恒等式：

- `I(n)=[n=1]` 是卷积单位元。

- `1(n)=1`，`id(n)=n`。
- `mu * 1 = I`，mu 是常数函数 `1` 的卷积逆。
- $\varphi * 1 = id$，$\varphi(n) = \sum_{d|n} mu(d) * (n/d)$。

9.2 约数和反演

若 $F(n) = \sum_{d|n} f(d)$，则 $f(n) = \sum_{d|n} mu(d)F(n/d)$。

这是最常见的莫比乌斯反演。

```
vector<ll> solve(const vector<ll>& F,const vector<int>& mu){
    int n=F.size()-1;
    vector<ll> f(n+1);
    for(int x=1;x<=n;x++)
        for(int d=1;1LL*d*x<=n;d++)
            f[x]+=1LL*mu[d]*F[x*d];
    return f;
}
```

9.3 倍数和反演

若 $F(n) = \sum_{k>=1} f(kn)$，则 $f(n) = \sum_{k>=1} mu(k)F(kn)$

```
vector<ll> solve(const vector<ll>& F,const vector<int>& mu){
    int n=F.size()-1;
    vector<ll> f(n+1);
    for(int d=1;d<=n;d++)
        for(int k=1;d*k<=n;k++)
            f[d]+=1LL*mu[k]*F[d*k];
    return f;
}
```

9.4 gcd 反演

设 `cnt[d]` 表示满足 `d | gcd(x,y)` 的对象数，`exact[g]` 表示满足 `gcd(x,y)=g` 的对象数：

$$cnt[d] = \sum_{k>=1} exact[kd]$$
$$exact[g] = \sum_{k>=1} mu(k)cnt[kg]$$

统计 gcd 恰好为 1 的数对

```
vector<ll> solve(const vector<int>& freq,const vector<int>& mu){
    int n=freq.size()-1;
    vector<ll> cnt(n+1),exact(n+1);
    for(int d=1;d<=n;d++){
        ll c=0;
        for(int x=d;x<=n;x+=d) c+=freq[x];
        cnt[d]=c*(c-1)/2;
    }
    for(int g=1;g<=n;g++)
        for(int k=1;g*k<=n;k++)
            exact[g]+=1LL*mu[k]*cnt[g*k];
    return exact;
}
```

最终 `exact[1]` 就是互质数对数量。

9.5 lcm 反演

设 `cnt[d]` 表示 `lcm(x,y)` 是 `d` 的倍数，`exact[l]` 表示 `lcm(x,y)=l`：

$cnt[d] = \sum_{d|l} exact[l]$
$exact[l] = \sum_{k>=1} mu[k] * cnt[lk]$

常见做法是先把 “lcm 是 `d` 的倍数” 转成 “`d` 同时整除两个数”，再做倍数反演。

9.6 欧拉函数反演

$$\sum_{d|n}\varphi(d)=n \Rightarrow \sum_{i=1}^{n}\gcd(i,n)=\sum_{d|n}\varphi(n)\frac{n}{d}$$

$$\varphi(n)=\sum_{d|n}mu(d)\frac{n}{d}$$

统计所有数对的 gcd 之和

由 $\gcd(i,j)=\sum_{d|\gcd(i,j)}\varphi(d)$，交换求和顺序：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}\gcd(i,j)=\sum_{d=1}^{\min(n,m)}\varphi(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor$$

```
long long solve(int n,int m,const vector<int>& phi){
    int N=min(n,m);
    long long ans=0;
    for(int d=1;d<=N;d++)
        ans+=1LL*phi[d]*(n/d)*(m/d);
    return ans;
}
```

9.7 二项式反演

若 $F(n)=\sum_{k=0}^{n}C(n,k)f(k)$，则 $f(n)=\sum_{k=0}^{n}(-1)^{n-k}C(n,k)F(k)$

反向也成立：

```
vector<ll> solve(const vector<ll>& F,int mod){
    int n=F.size()-1;
    vector<ll> fac(n+1,1),ifac(n+1,1),f(n+1);
    for(int i=1;i<=n;i++) fac[i]=fac[i-1]*i%mod;
    ifac[n]=qpow(fac[n],mod-2,mod);
    for(int i=n;i>=1;i--) ifac[i-1]=ifac[i]*i%mod;
    auto C=[&](int a,int b){
        if(b<0||b>a) return 0LL;
        return fac[a]*ifac[b]%mod*ifac[a-b]%mod;
    };
    for(int i=0;i<=n;i++) for(int k=0;k<=i;k++){
        ll term=C(i,k)*F[k]%mod;
        if((i-k)&1) f[i]=(f[i]-term+mod)%mod;
        else f[i]=(f[i]+term)%mod;
    }
    return f;
}
```

9.8 子集和反演

$F(S)=\sum_{T\subseteq S}f(T)$
$f(S)=\sum_{T\subseteq S}(-1)^{|S|-|T|}F(T)$

```
void subsetZeta(vector<ll>& f,int n){
    for(int bit=0;bit<n;bit++)
        for(int mask=0;mask<(1<<n);mask++)
            if(mask&(1<<bit)) f[mask]+=f[mask^(1<<bit)];
}
void subsetMobius(vector<ll>& f,int n){
    for(int bit=0;bit<n;bit++)
        for(int mask=0;mask<(1<<n);mask++)
            if(mask&(1<<bit)) f[mask]-=f[mask^(1<<bit)];
}
```

9.9 超集和反演

$F(S)=\sum_{T\supseteq S}f(T)$
$f(S)=\sum_{T\supseteq S}(-1)^{|T|-|S|}F(T)$

```
void supersetZeta(vector<ll>& f,int n){
    for(int bit=0;bit<n;bit++)
        for(int mask=0;mask<(1<<n);mask++)
            if(!(mask&(1<<bit))) f[mask]+=f[mask|(1<<bit)];
}
void supersetMobius(vector<ll>& f,int n){
    for(int bit=0;bit<n;bit++)
        for(int mask=0;mask<(1<<n);mask++)
            if(!(mask&(1<<bit))) f[mask]-=f[mask|(1<<bit)];
}
```

10. FFT/NTT

FFT

```
typedef complex<double> cd;
const double PI=acos(-1.0);
void fft(vector<cd>& a,bool invert){
    int n=a.size();
    for(int i=1,j=0;i<n;i++){
        int bit=n>>1;
        for(;j&bit;bit>>=1) j^=bit;
        j^=bit;
        if(i<j) swap(a[i],a[j]);
    }
    for(int len=2;len<=n;len<<=1){
        double ang=2*PI/len;
        if(invert) ang=-ang;
        cd wlen(cos(ang),sin(ang));
        for(int i=0;i<n;i+=len){
            cd w(1);
            for(int j=0;j<len/2;j++){
                cd u=a[i+j],v=w*a[i+j+len/2];
                a[i+j]=u+v; a[i+j+len/2]=u-v; w*=wlen;
            }
        }
    }
    if(invert)for(cd& x:a) x/=n;
}
vector<int> convolution(vector<int> a,vector<int> b){
    int n=1;
    while(n<(int)a.size()+(int)b.size()-1) n<<=1;
    vector<cd> fa(a.begin(),a.end()),fb(b.begin(),b.end());
    fa.resize(n);fb.resize(n); fft(fa,false);fft(fb,false);
    for(int i=0;i<n;i++) fa[i]*=fb[i];
    fft(fa,true);vector<int>res(n);
    for(int i=0;i<n;i++) res[i]=(int)round(fa[i].real());
    return res;//求得的卷积为res数组的[0...a.size()+b.size()]
}
```

NTT

模数为 998244353 的版本，求多项式 $A(x)\cdot B(x)$ 卷积，结果为卷积的每项系数

$$C_k=\sum_{i+j=k}A_i\times B_j(mod)$$

```
const int NTT_MOD=998244353, NTT_G=3;
void ntt(vector<int>& a,bool invert){
    int n=a.size();
    for(int i=1,j=0;i<n;i++){
        int bit=n>>1;for(;j&bit;bit>>=1)j^=bit;j^=bit;
        if(i<j)swap(a[i],a[j]);
    }
    for(int len=2;len<=n;len<<=1){
        int wlen=qpow(NTT_G,(NTT_MOD-1)/len,NTT_MOD);
        if(invert)wlen=qpow(wlen,NTT_MOD-2,NTT_MOD);
        for(int i=0;i<n;i+=len){
            long long w=1;
            for(int j=0;j<len/2;j++){
                int u=a[i+j],v=w*a[i+j+len/2]%NTT_MOD;
                a[i+j]=(u+v)%NTT_MOD;a[i+j+len/2]=(u-v+NTT_MOD)%NTT_MOD;
                w=w*wlen%NTT_MOD;
            }
        }
    }
    if(invert){int invn=qpow(n,NTT_MOD-2,NTT_MOD);for(int&
x:a)x=1LL*x*invn%NTT_MOD;}
}
vector<int> convolution(vector<int> a,vector<int> b){
    int n=1;while(n<(int)a.size()+(int)b.size()-1)n<<=1;
    a.resize(n);b.resize(n);ntt(a,false);ntt(b,false);
    for(int i=0;i<n;i++)a[i]=1LL*a[i]*b[i]%NTT_MOD;
    ntt(a,true);return a;
}
```

11. FWT

只放了 $\oplus$ 的版本：

$$C_k=\sum_{i\oplus j=k}A_i\times B_j$$

```
void fwtXor(vector<ll>& a,bool inv,ll mod){
    int n=a.size();
    for(int len=1;len<n;len<<=1)for(int i=0;i<n;i+=len<<1)for(int j=0;j<len;j++){
        ll x=a[i+j],y=a[i+j+len];
        a[i+j]=(x+y)%mod;a[i+j+len]=(x-y+mod)%mod;
    }
    if(inv){ll iv=qpow(n,mod-2,mod);for(ll& x:a)x=x*iv%mod;}
}
vector<ll> xorConvolution(vector<ll> a,vector<ll> b,ll mod){
    int n=1;while(n<(int)max(a.size(),b.size()))n<<=1;a.resize(n);b.resize(n);
    fwtXor(a,false,mod);fwtXor(b,false,mod);
    for(int i=0;i<n;i++)a[i]=a[i]*b[i]%mod;
```

```
    fwtXor(a,true,mod);return a;
}
```

七、动态规划 DP

1. 线性 DP / LIS

最长上升子序列

```
int LIS(vector<int> a){
    vector<int> d;
    for(int x:a){
        auto it=lower_bound(d.begin(),d.end(),x);
        if(it==d.end())d.push_back(x);else *it=x;
    }
    return d.size();
}
```

二维偏序最长链

思路：一维升序，另一维降序排序，再对第二维做 LIS，避免同第一维重复选。

```
int envelopeLIS(vector<pair<int,int>> a){
    sort(a.begin(),a.end(),[](auto&x,auto&y){return x.first==y.first?
x.second>y.second:x.first<y.first;});
    vector<int> b;for(auto [x,y]:a)b.push_back(y);return LIS(b);
}
```

2. 背包 DP

多重背包二进制拆分

```
vector<int>nw,nv;
for(int i=0;i<(int)w.size();i++)
    for(int k=1;c[i];k<<=1){
        int t=min(k,c[i]);c[i]-=t;
        nw.push_back(w[i]*t);nv.push_back(v[i]*t);
    }
return knapsack01(nw,nv,W);//做01背包
```

3. 区间 DP

石子合并

```
long long mergeStones(vector<int> a){
    int n=a.size();vector<ll> pre(n+1);for(int i=0;i<n;i++)pre[i+1]=pre[i]+a[i];
    vector<vector<ll>> dp(n,vector<ll>(n));
    for(int len=2;len<=n;len++)for(int l=0;l+len<=n;l++){
        int r=l+len-1;dp[l][r]=LINF;
        for(int k=l;k<r;k++)dp[l][r]=min(dp[l][r],dp[l][k]+dp[k+1][r]+pre[r+1]-
pre[l]);
    }
    return dp[0][n-1];
}
```

环形石子合并

```
// 把数组复制一遍，在长度为 n 的所有区间上跑区间 DP，答案取 min/max。
```

4. 树形 DP

树上最大独立集

选择尽可能多的节点，使得这些节点两两之间没有边相连

```
vector<vector<int>> g;
vector<array<int,2>> f;
void dfs(int u,int fa){
    f[u][0]=0;f[u][1]=1;
    for(int v:g[u])
        if(v!=fa){
            dfs(v,u);
            f[u][0]+=max(f[v][0],f[v][1]);
            f[u][1]+=f[v][0];
        }
}
```

换根 DP 求每点距离和

```
int sz[N],dp[N],ans[N];
void dfs1(int u,int p){
    sz[u]=1;
    for(int v:g[u]) if(v!=p){
        dfs1(v,u);sz[u]+=sz[v];
        dp[u]+=dp[v]+sz[v];
    }
}
void dfs2(int u,int p){
    for(int v:g[u]) if(v!=p){
        ans[v]=ans[u]-sz[v]+(n-sz[v]);
        dfs2(v,u);
    }
}
void solve(int n){
    dfs1(1,0);ans[1]=dp[1];dfs2(1,0);
}
```

5. 状压 DP

TSP 最短哈密顿路径

给定一系列城市和每对城市之间的距离，求解访问每一座城市一次并回到起始城市的最短回路

```
int tsp(vector<vector<int>> w){
    int n=w.size();
    vector<vector<int>> dp(1<<n,vector<int>(n,INF));
    dp[1][0]=0;
    for(int s=1;s<(1<<n);s++)for(int u=0;u<n;u++)if(dp[s][u]<INF){
        for(int v=0;v<n;v++)if(!(s>>v&1))dp[s|1<<v][v]=min(dp[s|1<<v][v],dp[s]
[u]+w[u][v]);
    }
    return *min_element(dp[(1<<n)-1].begin(),dp[(1<<n)-1].end());
}
```

集合划分

思路：预处理每个集合的代价 `cost[S]`，再枚举子集转移 `dp[S] = min(dp[S], dp[S^T] + cost[T])`。

```
for(int s=0;s<(1<<n);s++){
    for(int t=s;t;t=(t-1)&s){
        // dp[s] = min(dp[s], dp[s^t] + cost[t]);
    }
}
```

DAG 拓扑排序方案数

```
int n, m, son[N]; ll f[N];
int main() {
    cin>>n>>m;
    for(int i=1;i<=m;i++) {
        int x, y; cin>>x>>y;
        son[x] |= 1 << y - 1;
    } f[0] = 1;
    for(int s=0;s<=(1 << n) - 1;s++) for(int x=;x<=n;x++) {
        if ((son[x] & s) == son[x] && !(s >> x - 1 & 1)) f[s | (1 << x - 1)] +=
f[s];
    } cout<<f[(1 << n) - 1])<<'\n';
    return 0;
}
```

6. 数位 DP

统计 `[0,n]` 中不含数字 4 的数

```
ll memo[20][2];
ll dfs(int pos,bool tight,bool started){
```

```
    if(pos==m)return 1;
    if(!tight&&memo[pos][started]!=-1)return memo[pos][started];
    int up=tight?s[pos]-'0':9;ll ans=0;
    for(int d=0;d<=up;d++){
        if(d==4)continue;
        ans+=dfs(pos+1,tight&&d==up,started||d!=0);
    }
    if(!tight)memo[pos][started]=ans;
    return ans;
}
ll solve(ll n){
    string s=to_string(n);int m=s.size();
    memset(memo,-1,sizeof memo);
    return dfs(0,1,0);
}
```

7. DAG 上 DP

有向无环图路径方案数

```
vector<int> count(int n,int s){
    auto topo=topoSort(n,g);vector<int> dp(n+1);dp[s]=1;
    for(int u:topo)for(int v:g[u])dp[v]=(dp[v]+dp[u])%MOD;
    return dp;
}
```

8. 单调队列优化 DP

窗口最大值转移

```
vector<int> windowDp(vector<int> a,int k){
    int n=a.size();vector<int> dp(n);deque<int> q;
    for(int i=0;i<n;i++){
        while(!q.empty()&&q.front()<i-k)q.pop_front();
        dp[i]=a[i]+(q.empty()?0:dp[q.front()]);
        while(!q.empty()&&dp[q.back()]<=dp[i])q.pop_back();
        q.push_back(i);
    }
    return dp;
}
```

多重背包单调队列优化

思路：按余数分组，转移 `dp[j] = max(dp[j-k*w] + k*v)`，把 `dp[j-k*w]-k*v` 放进单调队列。

```
void boundedKnapsackMono(vector<int>& dp,int W,int w,int v,int c){
    for(int r=0;r<w;r++){
        deque<pair<int,int>> q;
        for(int k=0,j=r;j<=W;k++,j+=w){
            int val=dp[j]-k*v;
            while(!q.empty()&&q.back().second<=val)q.pop_back();
            q.push_back({k,val});
            while(!q.empty()&&k-q.front().first>c)q.pop_front();
            dp[j]=q.front().second+k*v;
        }
    }
}
```

9. 斜率优化 DP（CHT）

单调斜率队列

```
struct Line{ll k,b;ll y(ll x){return k*x+b;}};
bool bad(Line a,Line b,Line c){return (__int128)(b.b-a.b)*(a.k-c.k)>=(__int128)
(c.b-a.b)*(a.k-b.k);}
vector<ll> chtDp(vector<ll> x,vector<ll> k,vector<ll> b){
    deque<Line> q;vector<ll> ans(x.size());
    for(int i=0;i<(int)x.size();i++){
        Line nl{k[i],b[i]};
        while(q.size()>=2&&bad(q[q.size()-2],q.back(),nl))q.pop_back();
        q.push_back(nl);
        while(q.size()>=2&&q[0].y(x[i])>=q[1].y(x[i]))q.pop_front();
        ans[i]=q.front().y(x[i]);
    }
    return ans;
}
```

形如 `dp[i]=min(dp[j]+A[j]*B[i]+C[i])`

思路：把每个 `j` 看成一条直线 `y=A[j]*x+dp[j]`，查询 `x=B[i]` 的最小值。若斜率和查询点单调，可用队列；否则用 Li Chao Tree。

八、博弈论

1. Bash Game(巴什博弈)

> 一堆石子共n个，两人轮流从中取石子，规定每次至少取一个，最多取m个，最后取光者得胜。

```
bool bashWin(long long n,long long m){return n%(m+1)!=0;} // 每次取 1..m 个
cout << (bashWin(n,m) ? "First" : "Second") << '\n';
```

2. Wythoff Game(威佐夫博弈)

> 有两个石子堆，每次操作可以取走任意的一堆中任意多的石子或在两堆中同时取走相同数量的石子，不可以不取，无法操作者败。

令 $k=x-y$。若 $x=\lfloor \frac{1+\sqrt{5}}{2}k \rfloor$，先手必败。否则，先手必胜。

```
bool wythoffWin(long long a,long long b){
    if(a>b)swap(a,b);
    long long k=b-a;
    long long t=(long long)(k*(1+sqrt(5.0))/2);
    return t!=a;
}
```

3. Nim Game

> 地上有n堆石子，每人每次取一堆石子里任意多枚石子扔掉，可以取完，不能不取。最后没石子可取的人就输了。

```
bool nimWin(vector<long long> piles){
    long long x=0;for(ll v:piles)x^=v;return x!=0;
}
```

4. SG 函数

定义：对于状态 x 与其后继状态 $y_1,y_2,\ldots,y_n$，有
$SG(x)=mex(SG(y_1),SG(y_2),\ldots,SG(y_n))$

对于一个公平组合游戏，设其起点为 s，则 **当 $SG(s)\neq 0$ 时，先手必胜。**

若有多个起点 $s_1,s_2,\ldots,s_k$ 则当 $SG(s_1)\oplus SG(s_2)\oplus\cdots\oplus SG(s_k)\neq 0$ 时，先手必胜。同时，该组合游戏的 SG 值也为 $SG(s_1)\oplus SG(s_2)\oplus\cdots\oplus SG(s_k)\neq 0$。

```
vector<int> sgDAG(vector<vector<int>>& g){
    int n=g.size();vector<int> sg(n,-1);
    function<int(int)> dfs=[&](int u){
        if(sg[u]!=-1)return sg[u];
        set<int> s;for(int v:g[u])s.insert(dfs(v));
        int x=0;while(s.count(x))x++;return sg[u]=x;
    };
    for(int i=0;i<n;i++)dfs(i);return sg;
}
```

多个独立游戏和

```
// 总 SG = sg1 ^ sg2 ^ ...; 非零先手必胜，零先手必败。
```

九、计算几何

1. 向量

```
const double eps=1e-10;
inline int EPS(double x)
{
```

```
    if(fabs(x)<eps)return 0;
    return x<0?-1:1;
}
struct Vec{
    double x,y;
    Vec(double X=0,double Y=0){x=X;y=Y;}
    double len(){return sqrt(x*x+y*y);}
    Vec operator+(Vec a){return Vec(x+a.x,y+a.y);}
    Vec operator-(Vec a){return Vec(x-a.x,y-a.y);}
    Vec operator*(double a){return Vec(x*a,y*a);}
    Vec operator/(double a){return Vec(x/a,y/a);}
    double operator*(Vec a){return x*a.x+y*a.y;}//点积
    double operator%(Vec a){return x*a.y-y*a.x;}//叉积
    friend istream &operator>>(istream &cin,Vec &a){return cin>>a.x>>a.y;}
    friend ostream &operator<<(ostream &cout,Vec a){return
cout<<fixed<<setprecision(10)<<a.x<<' '<<setprecision(10)<<a.y;}
};
```

2.点和线

```
namespace Point_and_Segment{
    #define point Vec
    double Dis(Vec x,Vec y){return sqrt((x.x-y.x)*(x.x-y.x)+(x.y-y.y)*(x.y-
y.y));}
    bool On_Segment(point p1,point p2,point p0)//判断点在线段上
    {
        return EPS((p1-p0)%(p2-p0))==0&&EPS((p1-p0)*(p2-p0))<=0;
    }
    bool Intersect(point a,point b,point c,point d)//线段相交
    {
        if(max(a.x,b.x)<min(c.x,d.x)||max(c.x,d.x)<min(a.x,b.x)||max(a.y,b.y)
<min(c.y,d.y)||max(c.y,d.y)<min(a.y,b.y))return false;
        return EPS(((a-d)%(c-d))*((b-d)%(c-d)))<=0&&EPS(((d-b)%(a-b))*((c-b)%(a-
b)))<=0;
    }
    point Segment_Intersection(point a,point b,point c,point d)//线段交点
    {
        Vec ac=a-c,ab=b-a,cd=d-c,bc=b-c;
        double d1=fabs(ac%cd/cd.len());
        double d2=fabs(bc%cd/cd.len());
        double x=(d1*ab.len())/(d1+d2);
        Vec res=ab*x/ab.len();
        return res+a;
    }
    point Line_Intersection(point a,point b,point c,point d)//直线交点
    {
        double a1=b.y-a.y,b1=a.x-b.x,c1=b%a;
        double a2=d.y-c.y,b2=c.x-d.x,c2=d%c;
        return point((c2*b1-c1*b2)/(a1*b2-a2*b1),(c1*a2-c2*a1)/(a1*b2-a2*b1));
    }
    point Project_Point(point p1,point p2,point p0)//求p0到p1p2的投影点
    {
        Vec a=p2-p1,b=p0-p1;
        double res=a*b/a.len();
        Vec base=point(res*(p2.x-p1.x)/a.len(),res*(p2.y-p1.y)/a.len());
```

```
        return p1+base;
    }
    double Dis_point_to_segment(point a,point b,point c)//点到线段距离
    {
        Vec ab=b-a,ac=c-a,bc=c-b;
        double x=(ab*ac)*(ab*bc);
        if(EPS(x)>0)return min(ac.len(),bc.len());
        else return fabs((ab%ac)/ab.len());
    }
    double Dis_segment_to_segment(point a,point b,point c,point d)//线段到线段距离
    {
        if(Intersect(a,b,c,d))return 0;
        return
min({Dis_point_to_segment(a,b,c),Dis_point_to_segment(a,b,d),Dis_point_to_segment
(c,d,a),Dis_point_to_segment(c,d,b)});
    }
    #undef point
}using namespace Point_and_Segment;
```

3. 多边形

```
class Polygon: public vector<Vec>//多边形相关
{
    #define Point (*this)
    using point=Vec;
    public:
    double Area()//多边形面积
    {
        double res=0;point p=Point[0];
        for(int i=1;i<Point.size()-1;i++)
        {
            Vec a=Point[i]-p,b=Point[i+1]-p;
            res+=a%b;
        }
        return res/2;
    }
    double Diameter()//凸包直径
    {
        double ans=0;
        if(Point.size()==2)return Dis(Point[0],Point[1]);
        Polygon a=Point;a.emplace_back(a[0]);
        for(int i=0,j=1;i<a.size()-1;i++)
        {
            while((a[i+1]-a[i])%(a[j]-a[i])<(a[i+1]-a[i])%(a[j+1]-a[i]))
                j=(j+1)%(a.size()-1);
            ans=max({ans,Dis(a[i],a[j]),Dis(a[i+1],a[j])});
        }return ans;
    }
    double Convex_Cut(point x,point y)//凸切（凸包被直线所切的左边面积）
    {
        Polygon tmp;
        int n=Point.size();
        for(int i=0;i<n;i++)
        {
```

```
            Vec p1=x-Point[i],p2=y-Point[i],p3=x-Point[(i+1)%n],p4=y-
Point[(i+1)%n];
            if(EPS(p1%p2)!=-1)tmp.emplace_back(Point[i]);
            if(EPS(p1%p2)*EPS(p3%p4)<0)

tmp.emplace_back(Segment_Intersection(Point[i],Point[(i+1)%n],x,y));
        }
        double ans=0;
        for(int i=0;i<tmp.size();i++)
        {
            Vec p1=tmp[i],p2=tmp[(i+1)%tmp.size()];
            ans+=p1%p2;
        }
        return ans/2;
    }
    double Clostest_Pair()//求平面最近点对
    {
        sort(Point.begin(),Point.end(),[](const Vec &x,const Vec &y)
{if(x.x==y.x)return x.y<y.y;return x.x<y.x;});
        return Find_Clostest_Pair(0,Point.size()-1);
    }
    Polygon Convex_Hull_Andrew()//Andrew求凸包
    {
        Polygon a=Point,u,d;
        sort(a.begin(),a.end(),[](const Vec&x,const Vec &y){if(x.y==y.y)return
x.x<y.x;return x.y<y.y;});
        if(a.size()<=3)return a;
        #define IsRight(p1,p2,p0) EPS((p1-p0)%(p2-p0))
        for(auto x:a)
        {
            while(u.size()>=2&&IsRight(*(++u.rbegin()),*u.rbegin(),x)>=0)
                u.pop_back();
            u.emplace_back(x);
        }reverse(a.begin(),a.end());
        for(auto x:a)
        {
            while(d.size()>=2&&IsRight(*(++d.rbegin()),*d.rbegin(),x)>=0)
                d.pop_back();
            d.emplace_back(x);
        }
        #undef IsRight
        reverse(d.begin(),d.end());
        for(int i=u.size()-2;i>=1;i--)
            d.emplace_back(u[i]);
        return d;
    }
    int Point_In_Polygon(point p0)//求点是否在多边形内
    {
        bool flag=false;
        point p1,p2;
        for(int i=0,j=Point.size()-1;i<Point.size();j=i++)
        {
            p1=Point[i],p2=Point[j];
            if(On_Segment(p1,p2,p0))return 1;
            if((EPS(p1.y-p0.y)>0!=EPS(p2.y-p0.y)>0)&&EPS(p0.x-(p0.y-p1.y)*(p1.x-
p2.x)/(p1.y-p2.y)-p1.x)<0)
```

```
                flag=!flag;
        }return flag?2:0;
    }
    void print()
    {
        for(auto now:Point)
            cout<<now.x<<' '<<now.y<<'\n';
    }

    private:
    double Find_Clostest_Pair(int l,int r)
    {
        double dis=DBL_MAX;
        if(l==r)return dis;
        if(l+1==r)return Dis(Point[l],Point[r]);
        int mid=(l+r)>>1;
        double disl=Find_Clostest_Pair(l,mid);
        double disr=Find_Clostest_Pair(mid+1,r);
        dis=min(disl,disr);
        vector<int>tmp;
        for(int i=l;i<=r;i++)
            if(fabs(Point[mid].x-Point[i].x)<=dis)
                tmp.emplace_back(i);
        sort(tmp.begin(),tmp.end(),[&](const int &x,const int &y){return
Point[x].y<Point[y].y;});
        for(int i=0;i<tmp.size();i++)
        {
            for(int j=i+1;j<tmp.size()&&Point[tmp[j]].y-Point[tmp[i]].y<dis;j++)
            {
                double x=Dis(Point[tmp[i]],Point[tmp[j]]);
                if(x<dis)dis=x;
            }
        }
        vector<int>().swap(tmp);
        return dis;
    }
    #undef Point
};
```

补充：

1. Matrix-Tree 定理 (生成树的计数)

> 无向图生成树计数

思路：构造拉普拉斯矩阵，删去任意一行一列，剩余行列式就是生成树个数。重边自然会在度数和非对角线上累加。

```
ll detMod(vector<vector<ll>> a,ll mod){
    int n=a.size(); ll ans=1;
    for(int i=0;i<n;i++){
        int p=i;
        for(int j=i;j<n;j++) if(a[j][i]%mod){p=j;break;}
        if(!a[p][i]) return 0;
```

```
        if(p!=i) swap(a[p],a[i]), ans=(mod-ans)%mod;
        ans=ans*a[i][i]%mod;
        ll iv=qpow((a[i][i]%mod+mod)%mod,mod-2,mod);
        for(int j=i+1;j<n;j++){
            ll c=a[j][i]*iv%mod;
            for(int k=i;k<n;k++) a[j][k]=(a[j][k]-c*a[i][k])%mod;
        }
    }
    return (ans%mod+mod)%mod;
}
ll spanningTreeCount(int n,vector<pair<int,int>> edges,ll mod){
    vector<vector<ll>> lap(n-1,vector<ll>(n-1));
    for(auto [u,v]:edges){
        --u;--v;
        if(u<n-1) lap[u][u]++;
        if(v<n-1) lap[v][v]++;
        if(u<n-1&&v<n-1) lap[u][v]--,lap[v][u]--;
    }
    for(auto& row:lap) for(ll& x:row) x=(x%mod+mod)%mod;
    return detMod(lap,mod);
}
```

2. 高斯消元

实数线性方程组

```
int gauss(vector<vector<double>> a,vector<double>& x){
    int n=a.size(),m=a[0].size()-1,row=0;
    for(int col=0;col<m&&row<n;col++){
        int p=row;
        for(int i=row;i<n;i++) if(fabs(a[i][col])>fabs(a[p][col])) p=i;
        if(fabs(a[p][col])<1e-10) continue;
        swap(a[p],a[row]);
        for(int i=0;i<n;i++) if(i!=row){
            double c=a[i][col]/a[row][col];
            for(int j=col;j<=m;j++) a[i][j]-=c*a[row][j];
        }
        row++;
    }
    for(int i=row;i<n;i++) if(fabs(a[i][m])>1e-10) return 0; // 无解
    if(row<m) return 2; // 多解
    x.assign(m,0);
    for(int i=0;i<m;i++) x[i]=a[i][m]/a[i][i];
    return 1; // 唯一解
}
```